

Bloque 2 - Tema 1

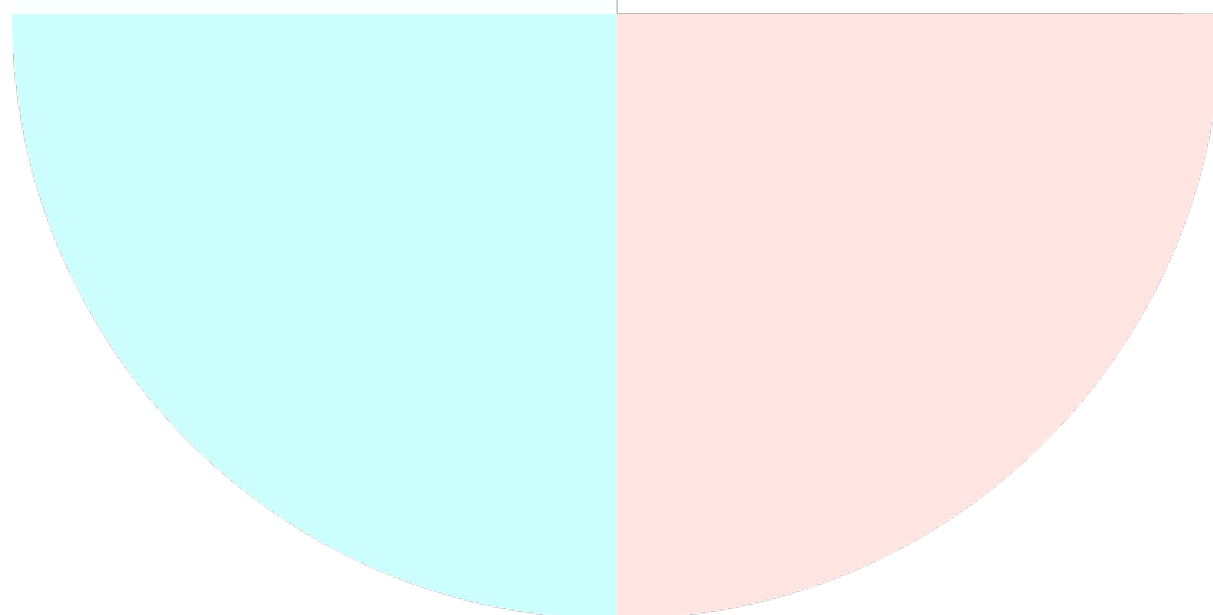
INFORMÁTICA BÁSICA. REPRESENTACIÓN Y COMUNICACIÓN DE LA INFORMACIÓN: ELEMENTOS CONSTITUTIVOS DE UN SISTEMA DE INFORMACIÓN. CARACTERÍSTICAS Y FUNCIONES. ARQUITECTURA DE ORDENADORES. COMPONENTES INTERNOS DE LOS EQUIPOS MICROINFORMÁTICOS

PREPARACIÓN OPOSICIONES

TÉCNICOS AUXILIARES DE INFORMÁTICA

ÍNDICE

ÍNDICE	2
1. INFORMÁTICA BÁSICA.....	3
2. REPRESENTACIÓN Y COMUNICACIÓN DE LA INFORMACIÓN	4
1. <i>Sistemas de numeración</i>	6
2. <i>Representación de textos</i>	10
3. <i>Funciones lógicas básicas</i>	12
3. ELEMENTOS CONSTITUTIVOS DE UN SISTEMA DE INFORMACIÓN	13
4. ARQUITECTURA DE ORDENADORES	15
1. <i>Organización del procesador</i>	15
2. <i>Arquitectura Von Neumann</i>	25
3. <i>Arquitectura Harvard</i>	31
4. <i>Taxonomía de Flynn</i>	31
5. <i>Arquitectura CISC vs RISC</i>	33
5. COMPONENTES INTERNOS DE LOS EQUIPOS MICROINFORMÁTICOS.....	36
1. <i>Placa base</i>	36
2. <i>Microprocesadores</i>	42
3. <i>Memoria interna</i>	44
4. <i>Proceso de arranque de un computador</i>	52



1. INFORMÁTICA BÁSICA

Entendemos por computador a una máquina o dispositivo capaz de realizar de manera automática y siguiendo un programa o secuencia de instrucciones, un determinado procesamiento sobre un conjunto de datos de entrada para obtener unos datos de salida.

Tradicionalmente la historia de la computación se ha dividido en 5 generaciones que sin estar delimitadas en el tiempo pretenden clasificar los computadores según la tecnología de construcción de las mismas y la forma en que el ser humano se comunica con ellas:

- **PRIMERA Generación:** desde la prehistoria hasta la década de los 40, la computación se basa en cálculos de forma más o menos automática, mediante tecnología mecánica. De los 40 los 50 se empiezan a usar tubos de vacío y relés electromagnéticos, se usa lenguaje binario y tarjetas perforadas. Máquinas enormes y muy costosas. Aparecen los primeros ordenadores comerciales.
- **SEGUNDA Generación:** de finales de los 50 a mediados de los 60. Computadoras diseñadas con circuitos de transistores. Máquinas más pequeñas y con mayor capacidad de procesamiento. Uso de lenguajes de alto nivel para la interfaz hombre-máquina.
- **TERCERA Generación:** de mediados de los 60 a principios de los 70. Evolución de la tecnología de los transistores que pasa a ser un único chip de silicio en un circuito integrado. Mayor velocidad de computación.
- **CUARTA Generación:** mediados de los 70. Aparecen los microprocesadores, que conlleva la disminución del tamaño de los computadores y mayor velocidad de procesamiento a coste muy reducido. Derivaron de ellos los PCs domésticos.
- **QUINTA Generación:** sería la actual, desde mediados de los 80. Marcada por la aparición de Internet. Se basa en investigaciones sobre la computación paralela y distribuida, la optimización de recursos, la miniaturización, el uso del lenguaje natural o la aplicación de inteligencia artificial.

Desde la primera generación, prácticamente todas las computadoras digitales se basan de una u otra forma en la arquitectura propuesta por Von Neumann, según la cual tanto los datos como los programas son almacenados en memoria antes de ser utilizados.

2. REPRESENTACIÓN Y COMUNICACIÓN DE LA INFORMACIÓN

BIT: unidad mínima de información, que puede tener solo dos valores (cero o uno). Acrónimo de *bi(nary) (digi)t* "dígito binario".

BYTE: conjunto de 8 bits que recibe el tratamiento de una unidad y que constituye el mínimo elemento de memoria direccionable de una computadora. Acrónimo formado por *bi(nary) te(rm)*. También se le denomina octeto.

PALABRA: conjunto de bits que constituyen la unidad de procesamiento en el ordenador. Se expresa en múltiplos enteros de byte. De esta forma, los datos e instrucciones que manejan las unidades del ordenador se miden en palabras.

NIBBLE: es la sucesión de 4 cifras binarias (bits), es decir, medio byte. Un nibble puede tomar uno de 2^4 valores posibles. Su importancia se debe a que 4 es el número mínimo de dígitos binarios necesarios para representar una cifra decimal. Además, un nibble representa exactamente un dígito hexadecimal.

CÚBIT: un cúbit o bit cuántico (quantum bit) es un sistema cuántico con dos estados propios y que puede ser manipulado arbitrariamente. Solo puede ser descrito correctamente mediante la mecánica cuántica. Los cúbits pueden representar numerosas combinaciones posibles de unos y ceros al mismo tiempo. La capacidad de estar simultáneamente en múltiples estados se llama *superposición cuántica*.

Unidades de capacidad

La capacidad (o tamaño de la memoria) hace referencia a la cantidad de información que se puede almacenar. La unidad utilizada para especificar la capacidad de almacenamiento de información es el byte (1 byte = 8 bits), y a la hora de indicar la capacidad, se utilizan diferentes prefijos que representan múltiplos del byte.

En el Sistema Internacional de medidas (SI) se utilizan prefijos que representan múltiplos y submúltiplos de una unidad; estos prefijos SI corresponden siempre a potencias de 10.

Cada prefijo del sistema internacional recibe un nombre diferente y utiliza un símbolo para representarlo. Los prefijos que utilizaremos más habitualmente son:

10^{-12}	pico (p)
10^{-9}	nano (n)
10^{-6}	micro (μ)
10^{-3}	mili (m)
10^3	kilo (K)
10^6	mega (M)
10^9	giga (G)
10^{12}	tera (T)
10^{15}	peta (P)
10^{18}	exa (E)
10^{21}	zetta (Z)
10^{24}	yotta (Y)
10^{27}	ronna (R)
10^{30}	quetta (Q)

Ejemplos:

10^3 bytes = 1.000 bytes = 1 Kilobyte (KB o Kbyte)

10^6 bytes = 10^3 KB = 1.000 KB = 1 Megabyte (MB o Mbyte)

10^9 bytes = 10^3 MB = 1.000 MB = 1 Gigabyte (GB o Gbyte)

10^{12} bytes = 10^3 GB = 1.000 GB = 1 Terabyte (TB o Tbyte)

Ahora bien, en informática, la capacidad de almacenamiento habitualmente se indica en múltiplos que sean potencias de 2; en este caso se utilizan los prefijos definidos en la norma ISO/IEC 80000-13.

B	Byte	B		1 B = 8 bits
K	Kibibyte	KiB	2^{10}	1 KiB = 2^{10} B
M	Mebibyte	MiB	2^{20}	1 MiB = 2^{10} KiB
G	Gibibyte	GiB	2^{30}	1 GiB = 2^{10} MiB
T	Tebibyte	TiB	2^{40}	1 TiB = 2^{10} GiB
P	Pebibyte	PiB	2^{50}	1 PiB = 2^{10} TiB

E	Exbibyte	EiB	2^{60}	1 EiB = 2^{10} PiB
Z	Zebibyte	ZiB	2^{70}	1 ZiB = 2^{10} EiB
Y	Yobibyte	YiB	2^{80}	1 YiB = 2^{10} ZiB
R	Robibyte	RiB	2^{90}	1 RiB = 2^{10} YiB
Q	Quebibyte	QiB	2^{100}	1 QiB = 2^{10} RiB

Robibyte y Quebibyte aún no han sido incorporados a la norma ISO

La industria utiliza mayoritariamente las unidades SI. Por ejemplo, si nos fijamos en las características de un disco duro que se comercialice con 1 TB de capacidad, realmente la capacidad del disco será de $1.000 \text{ GB} = 1.000.000 \text{ MB} = 1.000.000.000 \text{ KB} = 1.000.000.000.000$ bytes.

En cambio, cuando conectamos este disco a un ordenador y mostramos las propiedades del dispositivo, veremos que en la mayoría de los sistemas operativos se nos mostrará la capacidad en unidades IEC; en este caso, $976.562.500 \text{ KiB} = 953.674 \text{ MiB} = 931 \text{ GiB} = 0,91 \text{ TiB}$.

Los prefijos SI representan estrictamente potencias de 10. No deben utilizarse para expresar potencias de 2 (por ejemplo, un kilobit representa 1000 bits y no 1024 bits). Los prefijos adoptados por la IEC para las potencias binarias están publicados en la norma internacional IEC 60027-2: 2009, Símbolos literales a utilizar en electrotecnia - Parte 2: Telecomunicaciones y electrónica.

Los nombres y símbolos de los prefijos correspondientes a 2^{10} , 2^{20} , 2^{30} , 2^{40} , 2^{50} y 2^{60} son, respectivamente, kibi, Ki; mebi, Mi; gibi, Gi; tebi, Ti; pebi, Pi; y exbi, Ei. Así, por ejemplo, un kibibyte se escribe: $1 \text{ KiB} = 2^{10} \text{ B} = 1024 \text{ B}$, donde B representa al byte. Aunque estos prefijos no pertenecen al SI, deben emplearse en el campo de la tecnología de la información a fin de evitar un uso incorrecto de los prefijos SI¹.

1. Sistemas de numeración

Se define un sistema de numeración como el conjunto de símbolos y reglas que se utilizan para la representación de cantidades. En ellos existe un elemento característico que define el sistema y se denomina **base**, siendo el número de símbolos que se utilizan para la representación.

Un sistema de numeración en base "b" utiliza para representar los números un alfabeto compuesto por b símbolos o cifras. Así todo número se expresa por un conjunto de cifras, teniendo cada una de ellas dentro del número un valor que depende:

- De la cifra en sí.

¹ Real Decreto 2032/2009, de 30 de diciembre, por el que se establecen las unidades legales de medida

- De la **posición** que ocupe dentro del número.

Representación Número = $a_{n-1} a_{n-2} \dots a_2 a_1 a_0$

Valor de N = $a_{n-1} * b^{n-1} + a_{n-2} * b^{n-2} + \dots + a_2 * b^2 + a_1 * b^1 + a_0 * b^0$

Valor de a = Dígito x [Base sistema numérico]^{Posición relativa-1}

Existen diferentes sistemas de numeración para representar la información. Se caracterizan por tener una base que indica el número de símbolos a utilizar.

Sistema decimal (base 10)

Al ser b=10, el alfabeto está constituido por 10 símbolos: {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}.

Ejemplo: para el número 3483 aplicamos la fórmula anterior.

Valor de 3483 = $3 * 10^3 + 4 * 10^2 + 8 * 10^1 + 3 * 10^0$

Sistema binario (base 2)

Al ser b=2, el alfabeto está constituido por 2 símbolos: {0, 1}.

Cada dígito de un número representado en este sistema se denomina **bit** (binary digit).

Sistema octal (base 8)

Al ser b=8, el alfabeto está constituido por 8 símbolos: {0, 1, 2, 3, 4, 5, 6, 7}.

Sistema hexadecimal (base 16)

Al ser b=16, el alfabeto se compone de 16 símbolos: {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F}.

Además de ser potencia de dos, suelen emplear dos números hexadecimales para expresar el contenido de un **byte** (octeto).

Equivalencias entre sistemas de numeración

Decimal	Binario	Octal	Hexadecimal
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3

4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7
8	1000	(10)	8
9	1001	(11)	9
10	1010	(12)	A
11	1011	(13)	B
12	1100	(14)	C
13	1101	(15)	D
14	1110	(16)	E
15	1111	(17)	F

Conversión A sistema decimal

Utilizaremos la fórmula del recuadro para convertir un número de sistema binario/octal/hexadecimal a decimal.

- De sistema binario a sistema decimal

$$1101_2 = \underline{1} \cdot 2^3 + \underline{1} \cdot 2^2 + \underline{0} \cdot 2^1 + \underline{1} \cdot 2^0 = 8 + 4 + 0 + 1 = 13_{10}$$

$$1011101_2 = \underline{1} \cdot 2^6 + \underline{0} \cdot 2^5 + \underline{1} \cdot 2^4 + \underline{1} \cdot 2^3 + \underline{1} \cdot 2^2 + \underline{0} \cdot 2^1 + \underline{1} \cdot 2^0 = 64 + 16 + 8 + 4 + 1 = 93_{10}$$

- De sistema octal a sistema decimal

$$175_8 = \underline{1} \cdot 8^2 + \underline{7} \cdot 8^1 + \underline{5} \cdot 8^0 = 64 + 56 + 5 = 125_{10}$$

$$63_8 = \underline{6} \cdot 8^1 + \underline{3} \cdot 8^0 = 48 + 3 = 51_{10}$$

- De sistema hexadecimal a sistema decimal

$$C921_{16} = \underline{C} \cdot 16^3 + \underline{9} \cdot 16^2 + \underline{2} \cdot 16^1 + \underline{1} \cdot 16^0 = \underline{12} \cdot 4096 + 2304 + 32 + 1 = 51489_{10}$$

$$3AB_{16} = \underline{3} \cdot 16^2 + \underline{A} \cdot 16^1 + \underline{B} \cdot 16^0 = 768 + \underline{10} \cdot 16 + \underline{11} = 939_{10}$$

Ejercicios: Convertir a sistema decimal los siguientes números: FFA2₁₆, 110₁₆, EFA45₁₆, 345₈, 1101011101₂

Pregunta de examen

Indique de entre los siguientes números cuál es menor:

- a) 2A0 en sistema hexadecimal.
- b) 1226 en sistema octal.**
- c) 690 en sistema decimal.
- d) 1010110100 en sistema binario.

Explicación:

$$2A0_{16} = 2 \cdot 16^2 + 10 \cdot 16^1 = 512 + 160 = 672$$

$$1226_8 = 1 \cdot 8^3 + 2 \cdot 8^2 + 2 \cdot 8 + 6 = 512 + 128 + 16 + 6 = 662$$

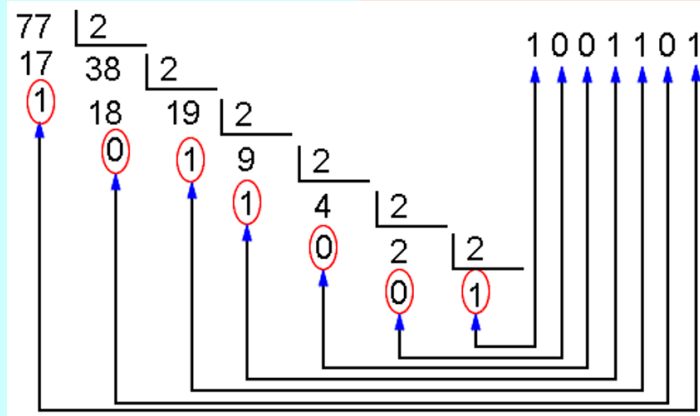
$$1010110100_2 = 1 \cdot 2^9 + 1 \cdot 2^7 + 1 \cdot 2^5 + 1 \cdot 2^4 + 1 \cdot 2^2 = 512 + 128 + 32 + 16 + 4 = 692$$

$$1226_8 < 2A0_{16} < 690_{10} < 1010110100_2$$

Conversión DESDE sistema decimal

- De sistema decimal a sistema binario: se obtiene efectuando divisiones enteras (sin obtener decimales) por 2, de la parte entera del número decimal de partida y de los cocientes que sucesivamente se vayan generando. Los restos de estas divisiones y el último cociente (que serán siempre ceros y unos) son las cifras binarias. El último cociente serán el bit más significativo y el primer resto el bit menos significativo (más a la derecha).

Ejemplo:



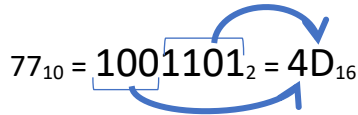
- De sistema decimal a sistema octal: como paso intermedio se obtiene el número en sistema binario y posteriormente se agrupan las cifras en grupos de 3 de derecha a izquierda.

Ejemplo: convertir a sistema octal el número 77_{10} .

$$77_{10} = 1001101_2 = 115_8$$

- De sistema decimal a sistema hexadecimal: como paso intermedio se obtiene el número en sistema binario y posteriormente se agrupan las cifras en grupos de 4 de derecha a izquierda.

Ejemplo: convertir a sistema hexadecimal el número 77_{10} .

$$77_{10} = 1001101_2 = 4D_{16}$$


Ejercicios: Convertir los siguientes números

125_{10} a sistema hexadecimal

99_{10} a sistema binario

146_{10} a sistema octal

345_{10} a sistema hexadecimal

283_{10} a sistema binario

2. Representación de textos

Cuando se introducen textos en un ordenador a través del periférico que corresponda, los caracteres se codifican con un código de entrada/salida, asociando a cada carácter una determinada combinación de n bits. Un código de E/S es por tanto una correspondencia entre el conjunto de caracteres y el alfabeto binario:

$$\alpha \equiv \{0, 1, 2, \dots, a, b, \dots, Y, Z, +, \&, \%, \dots\} \rightarrow \beta \equiv \{0, 1\}^*$$

Si utilizamos n bits para codificar m símbolos, el número mínimo de bits necesarios para codificar un conjunto de símbolos depende del cardinal de este conjunto:

- Con 2 bits ($n=2$) podemos hacer $2^2 = 4$ combinaciones, es decir, se podrían codificar 4 símbolos distintos ($m=4$).
- Con 3 bits ($n=3$) serían $2^3 = 8$ combinaciones, 8 símbolos ($m=8$), y así sucesivamente.
- Con n bits se podrían codificar $m = 2^n$ símbolos distintos.

Código EBCDIC

El código EBCDIC (Extended Binary Coded Decimal Interchange Code), desarrollado por IBM para sus mainframes, utiliza $n=8$ bits para representar cada carácter de un total de $m=256$ caracteres.

Código ASCII

El término hace referencia a una codificación fija de caracteres que asigna determinados códigos a caracteres imprimibles como letras, números y signos de puntuación y a caracteres de control no imprimibles. La codificación ASCII se utiliza para definir la forma como los

dispositivos electrónicos, como ordenadores o smartphones, representan visualmente estos caracteres.

El código ASCII desarrollado por el ANSI (American Standard Code for Information Interchange) es un código de **7 bits** con 128 caracteres (2^7) definidos, es decir, cuenta con 33 caracteres no imprimibles y 95 imprimibles y comprende tanto letras, signos de puntuación y números como caracteres de control.

El código ASCII se utiliza desde hace tiempo más para fines artísticos que para fines técnicos. El **ASCII Art** es un arte que solo emplea caracteres imprimibles de la tabla de ASCII para crear imágenes, cuya gama va desde trazos pasando por sencillas líneas patrón hasta verdaderos cuadros. Los artistas ASCII se valen así de las diferentes luminosidades de los caracteres para representar incluso matices.

ASCII extendido

Para hacer que el tamaño de cada patrón sea de **1 byte (8 bits)**, a los patrones de bits ASCII se les aumenta un 0 más a la izquierda. Ahora cada patrón puede caber fácilmente en un byte de memoria. En otras palabras, en **ASCII extendido** el primer patrón es **00000000** y el último es **01111111**.

Algunos fabricantes han decidido usar el bit de más para crear un sistema de 128 símbolos adicional. Sin embargo, este intento no ha tenido éxito debido a la secuencia no estándar creada por cada fabricante.

Unicode

Ninguno de los códigos anteriores representa símbolos que pertenecen a idiomas distintos al inglés. Por eso, se requiere un código con mucha más capacidad. Una coalición de fabricantes de hardware y software (Consortio Unicode) ha diseñado un código llamado **Unicode** (nombre que proviene de los 3 objetivos a alcanzar: universalidad, uniformidad y unicidad).

Así, Unicode es un estándar de codificación de caracteres (un total de 137.994) y los principios en los que se basó su diseño son la **universalidad**, la **eficiencia** y la **no ambigüedad**.

Unicode define cada carácter o símbolo mediante un nombre e identificador numérico, el **code point**. Diferentes secciones del código se asignan a los símbolos de distintos idiomas en el mundo. Algunas partes del código se usan para símbolos gráficos y especiales.

Unicode es mantenido por UTC (Comité Técnico Unicode, perteneciente al Consortio Unicode).

Los formatos de codificación que se pueden usar con Unicode son:

- **UTF-8 (8-bit Unicode Transformation Format)**: usa de 1 a 4 bytes por carácter (longitud variable), dependiendo del símbolo de Unicode. Los bits de carácter Unicode son divididos en varios grupos. Los caracteres más pequeños que 128_{10} son codificados con un byte

sencillo que contiene su valor: este corresponde exactamente a los caracteres de 7-bit de los 128 del ASCII.

- **UTF-16** (16-bit Unicode Transformation Format): utiliza 1 o 2 palabras de 16 bits (2 o 4 bytes) por carácter (longitud variable).
- **UTF-32** (32-bit Unicode Transformation Format): utiliza 32 bits (4 bytes) para codificar un símbolo (longitud fija).

3. Funciones lógicas básicas

NOT La salida es la negación de la entrada.

A	NOT A
0	1
1	0

AND La salida=1 si las dos entradas son 1, si no la salida=0.

A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1

NAND Invierte la salida de AND. La salida=0 si las dos entradas son 1, si no la salida=1.

A	B	A NAND B
0	0	1
0	1	1
1	0	1
1	1	0

OR La salida=1 si alguna o las dos entradas son 1, si no la salida=0.

A	B	A OR B
0	0	0
0	1	1
1	0	1
1	1	1

NOR Invierte la salida de OR. La salida=1 si las dos entradas son 0, si no la salida=0.

A	B	A NOR B
0	0	1
0	1	0
1	0	0
1	1	0

XOR La salida=1 si solo una entrada es 1, si no la salida=0 (suma con acarreo).

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

XNOR La salida=1 si las entradas son iguales, si no la salida=0.

A	B	A XNOR B
0	0	1
0	1	0
1	0	0
1	1	1

Pregunta de examen

En relación con las funciones lógicas básicas. Suponiendo que $a=0$ y $b=1$, ¿cuál de las siguientes sentencias es INCORRECTA?

- a) $a \text{ XOR } b = 1$.
c) $a \text{ NOR } b = 1$.

- b) $a \text{ XNOR } b = 0$.
d) $a \text{ NAND } b = 1$.

3. ELEMENTOS CONSTITUTIVOS DE UN SISTEMA DE INFORMACIÓN

Los Sistemas de Información fueron considerados inicialmente como un elemento que podía proporcionar ahorros de coste en las organizaciones, en la medida que podía dar soporte a actividades operativas en las que la información constituía el principal elemento implicado.

Hoy el Sistema de Información constituye la base para el desarrollo de nuevos productos o servicios, es el soporte principal del trabajo de los directivos, permite coordinar el trabajo dentro y entre organizaciones y sobre todo permite mejorar el funcionamiento, desarrollando nuevos modelos organizativos con una clara orientación a la información, coincidente con lo que hoy se conoce como “orientación a los procesos”.

Las tecnologías de la información facilitan los nuevos diseños organizativos a los que hacemos referencia, al tiempo que dan lugar a nuevas formas y procedimientos de gestión, nuevas estrategias y nuevos valores. El grado de cambio introducido en base a las tecnologías y los sistemas de información dependerá fundamentalmente de la forma en que éstas se apliquen, pudiendo convertirse en el verdadero motor del cambio y principal fuente de ventajas competitivas, si el proceso de mejora se gestiona adecuadamente.

Un **sistema de información** es un conjunto de componentes interrelacionados que recogen, procesan y distribuyen datos e información, incluyendo asimismo los mecanismos de retroalimentación implicados.

Un sistema de información está constituido por un enorme número de partes o subsistemas, que interaccionan unos con los otros en grado diferente. La estructuración de estas partes tiene simultáneamente una dimensión vertical y horizontal



En su **dimensión vertical**, el sistema de información tiene niveles jerárquicos que abordan los temas con diferentes grados de detalle. Se establece una división en tres niveles Estratégico, Táctico y Operativo.

- El **nivel OPERACIONAL** maneja procedimientos de rutina relacionados con actividades, tales como el tratamiento de los pedidos de clientes, preparación de documentos de embarque y facturación a clientes, etc. Los subsistemas operacionales recogen datos de los sucesos del mundo real y los almacenan en un depósito de datos (registro de fichas, base de datos, etc.).
- El **nivel TÁCTICO** del sistema de información trata de la toma de decisiones a corto plazo. La toma de decisiones táctica implica normalmente la intervención directa de una persona. Bastante información en apoyo a la decisión proviene de la base de datos y casi siempre requiere la confección de algún tipo de cálculo para obtener resúmenes o hacer algún tipo de proyección hacia el futuro (previsiones, etc.).
- El **nivel ESTRATÉGICO** es similar al nivel táctico, excepto en que trata decisiones más amplias. Normalmente las decisiones de carácter estratégico se suelen tomar a medio y largo plazo y son más difíciles de formalizar que las decisiones de nivel operativo, requiriendo normalmente una información más elaborada, que aporte una visión integrada de la empresa.

En su **dimensión horizontal**, cada nivel puede dividirse en subsistemas (especialmente en el nivel operativo y táctico), que pueden estar directamente conectados unos con los otros, aportando un alto grado de integración con un considerable aumento de eficiencia. Esta situación comienza a ser frecuente en las empresas, debido a la posibilidad ofrecida por los sistemas informáticos para compartir la información entre diversos usuarios y aplicaciones.

Por último, los **componentes** o elementos que precisa cualquier sistema de información son:

- **Hardware:** equipamiento informático.
- **Software:** conjunto de programas que conforman el sistema lógico.
- **Bases de datos:** almacena la información de la organización.
- **Comunicaciones:** permite la interconexión e integración con otros sistemas.
- **Personas:** es el recurso más valioso, formado por personal TIC y usuarios.
- **Procedimientos:** incluye las estrategias, políticas, métodos y reglas que en general se aplican en el uso y gestión del sistema de información.

4. ARQUITECTURA DE ORDENADORES

1. Organización del procesador

La función principal de un procesador es ejecutar instrucciones y la organización que tiene viene condicionada por las tareas que debe realizar y por cómo debe hacerlo.

Los procesadores están diseñados y operan según una señal de sincronización. Esta señal, conocida como **señal de reloj**, es una señal en forma de onda cuadrada periódica con una determinada frecuencia. Todas las operaciones hechas por el procesador las gobierna esta señal de reloj: un ciclo de reloj determina la unidad básica de tiempo, es decir, la duración mínima de una operación del procesador.

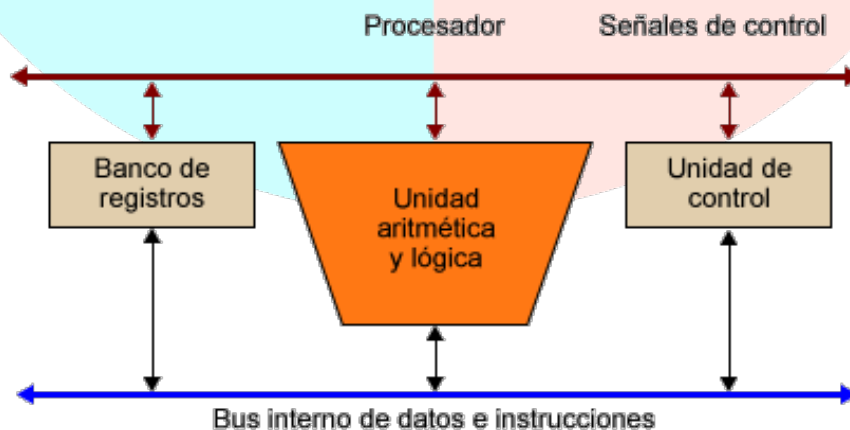
Para ejecutar una instrucción, son necesarios uno o más ciclos de reloj, dependiendo del tipo de instrucción y de los operandos que tenga.

Las prestaciones del procesador no las determina solo la frecuencia de reloj, sino otras características del procesador, especialmente del diseño del juego de instrucciones y la capacidad que tiene para ejecutar simultáneamente múltiples instrucciones.

Para ejecutar las instrucciones, todo procesador dispone de tres componentes principales:

- 1) **Un conjunto de registros:** espacio de almacenamiento temporal de datos e instrucciones dentro del procesador.
- 2) **Unidad aritmética y lógica o ALU:** circuito que hace un conjunto de operaciones aritméticas y lógicas con los datos almacenados dentro del procesador.
- 3) **Unidad de control:** circuito que controla el funcionamiento de todos los componentes del procesador. Controla el movimiento de datos e instrucciones dentro y fuera del procesador y también las operaciones de la ALU.

La organización básica de los elementos que componen un procesador y el flujo de información entre los diferentes elementos se ve en el esquema siguiente:



Como se observa, aparte de los tres componentes principales, es necesario disponer de un sistema que permita interconectar estos componentes. Este sistema de interconexión es específico para cada procesador. Distinguimos dos tipos de líneas de interconexión: líneas de control, que permiten gobernar el procesador, y líneas de datos, que permiten transferir los datos y las instrucciones entre los diferentes componentes del procesador. Este sistema de interconexión tiene que disponer de una interfaz con el bus del sistema.

El término **procesador** actualmente se puede entender como microprocesador porque todas las unidades funcionales que forman el procesador se encuentran dentro de un chip, pero hay que tener presente que, por el aumento de la capacidad del nivel de integración, dentro de los microprocesadores se pueden encontrar otras unidades funcionales del computador. Por ejemplo:

- **Unidades de ejecución SIMD:** unidades especializadas en la ejecución de instrucciones SIMD (Single Instruction, Multiple Data), instrucciones que trabajan con estructuras de datos vectoriales, como por ejemplo instrucciones multimedia.
- **Memoria caché:** prácticamente todos los procesadores modernos incorporan dentro del propio chip del procesador algunos niveles de memoria caché.
- **Unidad de gestión de memoria o Memory Management Unit (MMU):** gestiona el espacio de direcciones virtuales, traduciendo las direcciones de memoria virtual a direcciones de memoria física en tiempo de ejecución. Esta traducción permite proteger el espacio de direcciones de un programa del espacio de direcciones de otros programas y también permite separar el espacio de memoria del sistema operativo del espacio de memoria de los programas de usuario.
- **Unidad de punto flotante o Floating Point Unit (FPU):** unidad especializada en hacer operaciones en punto flotante; puede funcionar de manera autónoma, ya que dispone de un conjunto de registros propio.

Instrucciones

Las instrucciones de una arquitectura son autocontenidas, es decir, incluyen toda la información necesaria para su ejecución.

Los elementos que componen una instrucción, independientemente del tipo de arquitectura, son los siguientes:

Codop	Op fuente 1	Op fuente 2	...	Op fuente N	Op destino	Dir. Instrucción siguiente
-------	-------------	-------------	-----	-------------	------------	----------------------------

- **Código de operación:** especifica la operación que hace la instrucción.
- **Operandos fuente:** para hacer la operación pueden ser necesarios uno o más operandos fuente; uno o más de estos operandos pueden ser implícitos.
- **Operando destino:** almacena el resultado de la operación realizada. Puede estar explícito o implícito. Uno de los operandos fuente se puede utilizar también como operando destino.

- **Dirección de la instrucción siguiente:** especifica dónde está la instrucción siguiente que se debe ejecutar. Suele ser una información implícita, ya que el procesador va a buscar automáticamente la instrucción que se encuentra a continuación de la última instrucción ejecutada. Solo las instrucciones de ruptura de secuencia especifican una dirección alternativa.

Uno de los elementos necesarios en cualquier instrucción es el **conjunto de operandos**. Los operandos necesarios para ejecutar una instrucción pueden encontrarse explícitamente en la instrucción o pueden ser implícitos.

La localización dentro del procesador de los operandos necesarios para ejecutar una instrucción y la manera de explicitarlos dan lugar a arquitecturas diferentes del juego de instrucciones.

Según los criterios de localización y la explicitación de los operandos, podemos identificar las arquitecturas del juego de instrucciones siguientes:

- **Arquitecturas basadas en PILA:** los operandos son implícitos y se encuentran en la pila.
- **Arquitecturas basadas en ACUMULADOR:** uno de los operandos se encuentra de manera implícita en un registro denominado *acumulador*.
- **Arquitecturas basadas en REGISTROS de propósito general:** los operandos se encuentran siempre de manera explícita, ya sea en registros de propósito general o en la memoria.

Ahora bien, dentro de las arquitecturas basadas en registros de propósito general, podemos distinguir tres subtipos:

- **Registro-registro (o load-store).** Solo pueden acceder a la memoria instrucciones de carga (*load*) y almacenamiento (*store*).
- **Registro-memoria.** Cualquier instrucción puede acceder a la memoria con uno de sus operandos.
- **Memoria-memoria.** Cualquier instrucción puede acceder a la memoria con todos sus operandos.

El subtipo registro-registro presenta las ventajas siguientes respecto a los registro-memoria y memoria-memoria: la codificación de las instrucciones es muy simple y de longitud fija, y utiliza un número parecido de ciclos de reloj del procesador para ejecutarse. Como contrapartida, se necesitan más instrucciones para hacer la misma operación que en los otros dos subtipos.

Los subtipos registro-memoria y memoria-memoria presentan las ventajas siguientes respecto al registro-registro: no hay que cargar los datos en registros para operar en ellos y facilitan la programación. Como contrapartida, los accesos a memoria pueden crear cuellos de botella y el número de ciclos de reloj del procesador necesarios para ejecutar una instrucción puede variar mucho, lo que dificulta el control del ciclo de ejecución y lentifica su ejecución.

Podemos hablar de 5 **tipos de arquitectura** del juego de instrucciones:

- 1) Pila.
- 2) Acumulador.
- 3) Registro-registro.
- 4) Registro-memoria.
- 5) Memoria-memoria

Operandos

Los operandos de una instrucción pueden expresar directamente un dato, la dirección, o la referencia a la dirección, donde tenemos el dato. Esta dirección puede ser la de un registro o la de una posición de memoria, y en este último caso la denominaremos **dirección efectiva**.

Se define **modo de direccionamiento** como las diferentes maneras de expresar un operando en una instrucción y el procedimiento asociado que permite obtener la dirección donde está almacenado el dato y, como consecuencia, el dato.

Los modos de direccionamiento más comunes son:

- Direccionamiento **inmediato**.
- Direccionamiento **directo**:
 - o Direccionamiento directo a registro.
 - o Direccionamiento directo a memoria.
- Direccionamiento **indirecto**:
 - o Direccionamiento indirecto a registro.
 - o Direccionamiento indirecto a memoria.
- Direccionamiento **relativo**:
 - o Direccionamiento relativo a registro base o relativo.
 - o Direccionamiento relativo a registro índice (RI) o indexado.
 - o Direccionamiento relativo a PC.
- Direccionamiento **implícito**:
 - o Direccionamiento a pila (indirecto a registro SP).

Es importante destacar el concepto **endian**, que expresa el orden de los bytes de un operando en la memoria. Existen 2 modos:

- **Big-endian**: almacena el byte más significativo del escalar en la dirección más baja de memoria.
- **Little-endian**: almacena el byte más significativo del escalar en la dirección más alta de memoria.

Ejemplo: el hexadecimal 12 34 56 78 almacenado en la dirección de memoria 184.

	BIG-ENDIAN	LITTLE-ENDIAN
Dirección	Dato	Dato
184	12	78
185	34	56
186	56	34
187	78	12

Ejecución de instrucciones

El ciclo de ejecución es la secuencia de operaciones que se hace para ejecutar cada una de las instrucciones. Lo dividimos en 4 fases principales:

- F1) Lectura de la instrucción.
- F2) Lectura de los operandos fuente.
- F3) Ejecución de la instrucción y almacenamiento del operando de destino.
- F4) Comprobación de interrupciones.

En la literatura también podemos encontrar la división del ciclo de ejecución en dos superfases:

- Fase de búsqueda (fetch).
- Fase de ejecución.

En cada fase se incluyen una o varias operaciones que hay que hacer. Para llevar a cabo estas operaciones, nombradas *microoperaciones*, es necesaria una compleja gestión de las señales de control y de la disponibilidad de los diferentes elementos del procesador, tarea realizada por la unidad de control.

Veamos las operaciones principales que se realizan habitualmente en cada una de las cuatro fases del ciclo de ejecución de las instrucciones:

Fase 1. Lectura de la instrucción. Las operaciones realizadas en esta fase son las siguientes:

- a) **Leer la instrucción.** Cuando leemos la instrucción, el registro contador del programa (PC) nos indica la dirección de memoria donde está la instrucción que hemos de leer. Si el tamaño de la instrucción es superior a la palabra de la memoria, hay que hacer tantos accesos a la memoria como sean necesarios para leerla completamente y cargar toda esta información en el registro de instrucción (IR).
- b) **Descodificar la instrucción.** Se identifican las diferentes partes de la instrucción para determinar qué operaciones hay que hacer en cada fase del ciclo de ejecución. Esta tarea

la realiza la unidad de control del procesador leyendo la información que hemos cargado en el registro de instrucción (IR).

- c) **Actualizar el contador del programa.** El contador del programa se actualiza según el tamaño de la instrucción, es decir, según el número de accesos a la memoria que hemos hecho para leer la instrucción.

Las operaciones de esta fase son comunes a los diferentes tipos de instrucciones que encontramos en el juego de instrucciones.

Fase 2. Lectura de los operandos fuente. Esta fase se debe repetir para todos los operandos fuente que tenga la instrucción.

Las operaciones que hay que realizar en esta fase dependen del modo de direccionamiento que tengan los operandos: para los más simples, como el inmediato o el directo a registro, no hay que hacer ninguna operación; para los indirectos o los relativos, hay que hacer cálculos y accesos a memoria. Si el operando fuente es implícito, vamos a buscar el dato en el lugar predeterminado por aquella instrucción.

Es fácil darse cuenta de que dar mucha flexibilidad a los modos de direccionamiento que podemos utilizar en cada instrucción puede afectar mucho al tiempo de ejecución de una instrucción.

Por ejemplo, si efectuamos un `ADD R3,3` no hay que hacer ningún cálculo ni ningún acceso a memoria para obtener los operandos; en cambio, si efectuamos un `ADD [R1], [R2+16]`, hay que hacer una suma y dos accesos a memoria.

Fase 3. Ejecución de la instrucción y almacenamiento del operando de destino (resultado)

Las operaciones que hay que realizar en esta fase dependen del código de operación de la instrucción y del modo de direccionamiento que tenga el operando destino.

- a) **Ejecución de la instrucción:** Las operaciones que se llevan a cabo son diferentes para cada código de operación. Durante la ejecución, además de obtener el resultado de la ejecución de la instrucción, se pueden modificar los bits de resultado de la palabra de estado del procesador.
- b) **Almacenamiento del operando de destino (resultado):** La función básica es recoger el resultado obtenido durante la ejecución y guardarlo en el lugar indicado por el operando, a menos que el operando sea implícito, en cuyo caso se guarda en el lugar predeterminado por aquella instrucción. El operando de destino puede ser uno de los operandos fuente; de esta manera, no hay que repetir los cálculos para obtener la dirección del operando.

Fase 4. Comprobación de interrupciones

Las interrupciones son el mecanismo mediante el cual un dispositivo externo al procesador puede interrumpir el programa que está ejecutando el procesador con el fin de ejecutar otro

programa (una rutina de servicio a la interrupción o RSI) para dar servicio al dispositivo que ha producido la interrupción.

La petición de interrupción se efectúa activando alguna de las líneas de petición de las que dispone el procesador.

En esta fase se verifica si se ha activado alguna línea de petición de interrupción del procesador en el transcurso de la ejecución de la instrucción. Si no se ha activado ninguna, continuamos el proceso normalmente; es decir, se acaba la ejecución de la instrucción en curso y se empieza la ejecución de la instrucción siguiente. En caso contrario, hay que transferir el control del procesador a la rutina de servicio de interrupción que debe atender esta interrupción y hasta que no se acabe no podemos continuar la ejecución de la instrucción en curso.

Para atender la interrupción, es necesario llevar a cabo algunas operaciones (intentaremos almacenar la mínima información para que el proceso sea tan rápido como se pueda) para transferir el control del procesador a la rutina de servicio de interrupciones, de manera que, después, lo podamos recuperar con la garantía de que el procesador estará en el mismo estado en el que estaba antes de transferir el control a la rutina de servicio de interrupción:

- Almacenar el contador del programa y la palabra de estado (generalmente, se guardan en la pila).
- Almacenar la dirección donde empieza la rutina para atender la interrupción en el contador del programa.
- Ejecutar la rutina de servicio de interrupción.
- Recuperar el contador del programa y la palabra de estado.

Esta fase actualmente está presente en todos los computadores, ya que todos utilizan E/S para interrupciones y E/S para DMA.

Segmentación de instrucciones

La segmentación de las instrucciones (**pipeline**) consiste en dividir el ciclo de ejecución de las instrucciones en un conjunto de etapas. Estas etapas pueden coincidir o no con las fases del ciclo de ejecución de las instrucciones.

El objetivo de la segmentación es ejecutar simultáneamente diferentes etapas de distintas instrucciones, lo cual permite aumentar el rendimiento del procesador sin tener que hacer más rápidas todas las unidades del procesador (ALU, UC, buses, etc.) y sin tener que duplicarlas.

La división de la ejecución de una instrucción en diferentes etapas se debe realizar de tal manera que cada etapa tenga la misma duración, generalmente un ciclo de reloj. Es necesario añadir registros para almacenar los resultados intermedios entre las diferentes etapas, de modo que la información generada en una etapa esté disponible para la etapa siguiente.

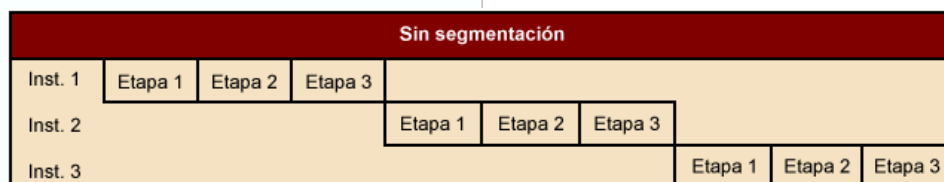
Ejemplo:

La segmentación es como una cadena de montaje. En cada etapa de la cadena se lleva a cabo una parte del trabajo total y cuando se acaba el trabajo de una etapa, el producto pasa a la siguiente y así sucesivamente hasta llegar al final.

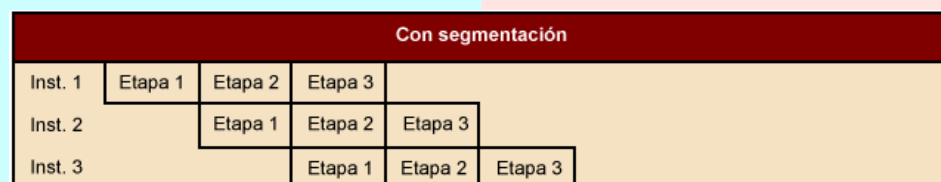
Si hay N etapas, se puede trabajar sobre N productos al mismo tiempo y, si la cadena está bien equilibrada, saldrá un producto acabado en el tiempo que se tarda en llevar a cabo una de las etapas. De esta manera, no se reduce el tiempo que se tarda en hacer un producto, sino que se reduce el tiempo total necesario para hacer una determinada cantidad de productos porque las operaciones de cada etapa se efectúan simultáneamente.

Si consideramos que el producto es una instrucción y las etapas son cada una de las fases de ejecución de la instrucción, que llamamos **etapa de segmentación**, hemos identificado los elementos y el funcionamiento de la segmentación.

Veámoslo gráficamente. Presentamos un mismo proceso con instrucciones de tres etapas, en el que cada etapa tarda el mismo tiempo en ejecutarse, resuelto sin segmentación y con segmentación:



Son necesarias 9 etapas para ejecutar las 3 instrucciones.



Son necesarias 5 etapas para ejecutar las 3 instrucciones.

Se ha reducido el tiempo total que se tarda en ejecutar las tres instrucciones, pero no el tiempo que se tarda en ejecutar cada una de las instrucciones.

Registros

Los registros son, básicamente, elementos de memoria de acceso rápido que se encuentran dentro del procesador. Constituyen un espacio de trabajo para el procesador y se utilizan como un espacio de almacenamiento temporal. Se implementan utilizando elementos de memoria RAM estática (*static RAM*). Son imprescindibles para ejecutar las instrucciones, entre otros motivos, porque la ALU solo trabaja con los registros internos del procesador.

El conjunto de registros y la organización que tienen cambia de un procesador a otro; los procesadores difieren en el número de registros, en el tipo de registros y en el tamaño de cada registro.

Una parte de los registros pueden ser visibles para el programador de aplicaciones, otra parte solo para instrucciones privilegiadas y otra solo se utiliza en el funcionamiento interno del procesador.

Una posible clasificación de los registros del procesador es la siguiente:

- (1) Registros de propósito general.
- (2) Registros de instrucción.
- (3) Registros de acceso a memoria.
- (4) Registros de estado y de control.

(1) Registros de propósito general

Los registros de propósito general son los registros que suelen utilizarse como operandos en las instrucciones del ensamblador. Estos registros se pueden asignar a funciones concretas: datos o direccionamiento. En algunos procesadores todos los registros se pueden utilizar para todas las funciones.

Los registros de datos se pueden diferenciar por el formato y el tamaño de los datos que almacenan. Puede haber registros para números enteros y para números en punto flotante.

También tenemos el registro “acumulador”, donde se almacenan temporalmente los datos que serán tratados por la Unidad Aritmética Lógica (ALU).

Los registros de direccionamiento se utilizan para acceder a memoria y pueden almacenar direcciones o índices. Algunos de estos registros se utilizan de manera implícita para diferentes funciones, como por ejemplo acceder a la pila, dirigir segmentos de memoria o hacer de soporte en la memoria virtual.

(2) Registros de instrucción

Los 2 registros principales relacionados con el acceso a las instrucciones son:

- **Program counter (PC)**: registro contador del programa, contiene la dirección de la instrucción siguiente que hay que leer de la memoria.
- **Instruction register (IR)**: registro de instrucción, contiene la instrucción que hay que ejecutar.

(3) Registros de acceso a memoria

Hay 2 registros necesarios para cualquier operación de lectura o escritura de memoria:

- **Memory Address Register (MAR)**: registro de direcciones de memoria, donde ponemos la dirección de memoria a la que queremos acceder.
- **Memory Buffer Register (MBR)**: registro de datos de memoria; registro donde la memoria deposita el dato leído o el dato que queremos escribir.

La manera de acceder a memoria utilizando estos registros es la siguiente:

- 1) En una **operación de lectura**, se realiza la secuencia de operaciones siguiente:
 - a) El procesador carga en el registro MAR la dirección de la posición de memoria que se quiere leer.
 - b) El procesador coloca en las líneas de direcciones del bus el contenido del MAR y activa la señal de lectura de la memoria.
 - c) El MBR se carga con el dato obtenido de la memoria.
- 2) En una **operación de escritura**, se realiza la secuencia de operaciones siguiente:
 - a) El procesador carga en el registro MBR la palabra que quiere escribir en la memoria.
 - b) El procesador carga en el registro MAR la dirección de la posición de memoria donde se quiere escribir el dato.
 - c) El procesador coloca en las líneas de direcciones del bus el contenido del MAR y en las líneas de datos del bus, el contenido del MBR, y activa la señal de escritura de la memoria.

(4) Registros de estado y de control

La información sobre el estado del procesador puede estar almacenada en un registro o en más de uno, aunque habitualmente suele ser un único registro denominado **registro de estado**.

Los bits del registro de estado son modificados por el procesador como resultado de la ejecución de algunos tipos de instrucciones, por ejemplo, instrucciones aritméticas o lógicas, o como consecuencia de algún acontecimiento, como las peticiones de interrupción. Estos bits son parcialmente visibles para el programador, en algunos casos mediante la ejecución de instrucciones específicas.

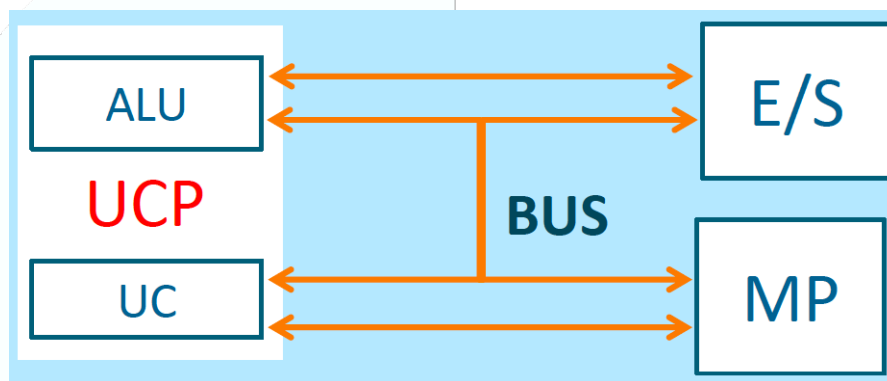
Cada bit o conjunto de bits del registro de estado indica una información concreta. Los más habituales son:

- **Bit de cero:** se activa si el resultado obtenido es 0.
- **Bit de transporte:** se activa si en el último bit que operamos en una operación aritmética se produce transporte; también puede deberse a una operación de desplazamiento.
- **Bit de desbordamiento:** se activa si la última operación ha producido un resultado que no se puede representar en el formato que estamos utilizando.
- **Bit de signo:** se activa si el resultado obtenido es negativo.
- **Bit de interrupción:** indica si las interrupciones están habilitadas o inhibidas.
- **Bit de modo de operación:** indica si la instrucción se ejecuta en modo supervisor o en modo usuario. Hay instrucciones que solo se ejecutan en modo supervisor.

- **Nivel de ejecución:** indica el nivel de privilegio de un programa en ejecución. Un programa puede desalojar el programa que se ejecuta actualmente si su nivel de privilegio es superior.

Los **registros de control** son los que dependen más de la organización del procesador. En estos registros se almacena la información generada por la unidad de control y también información específica para el sistema operativo. La información almacenada en estos registros no es nunca visible para el programador de aplicaciones.

2. Arquitectura Von Neumann



CPU = UC + ALU
Ordenador central = CPU + MP
Periféricos = E/S, Memoria Secundaria

PROCESADOR O CPU (UNIDAD CENTRAL DE PROCESO)

Es el responsable de la lectura y ejecución de las instrucciones almacenadas en memoria principal, realizando las operaciones sobre los datos almacenados en la memoria principal, generando nuevos datos que también serán almacenados en dicha memoria.

La CPU está compuesta por la UC, la ALU y los registros.

UC (UNIDAD DE CONTROL)

Contiene la instrucción que se está ejecutando en ese momento. Se encarga de generar las señales de control para la ejecución de la instrucción. Entre las partes de una instrucción destaca el código de operación y las direcciones de memoria donde se encuentran los operandos que pueda necesitar la instrucción.

La unidad de control se puede considerar el cerebro del computador. Como el cerebro, está conectada al resto de los componentes del computador mediante las señales de control (el sistema nervioso del computador). Con este símil no se pretende humanizar los

computadores, sino ilustrar que la UC es imprescindible para coordinar los diferentes elementos que tiene el computador y hacer un buen uso de ellos.

Es muy importante que un computador tenga unidades funcionales muy eficientes y rápidas, pero si no se coordinan y no se controlan correctamente, es imposible aprovechar todas las potencialidades que se habían previsto en el diseño.

Consiguientemente, muchas veces, al implementar una UC, se hacen evidentes las relaciones que hay entre las diferentes unidades del computador y nos damos cuenta de que hay que rediseñarlas, no para mejorar el funcionamiento concreto de cada unidad, sino para mejorar el funcionamiento global del computador.

La función básica de la UC es la ejecución de las instrucciones, pero su complejidad del diseño no se debe a la complejidad de estas tareas (que en general son muy sencillas), sino a la sincronización que se debe hacer de ellas.

Aparte de ver las maneras más habituales de implementar una UC, analizaremos el comportamiento dinámico, que es clave en la eficiencia y la rapidez de un computador.

Microoperaciones

Ejecutar un programa consiste en ejecutar una secuencia de instrucciones, y cada instrucción se lleva a cabo mediante un ciclo de ejecución que consta de las 4 fases principales antes vistas: **(1)** Lectura de la instrucción, **(2)** Lectura de los operandos fuente, **(3)** Ejecución de la instrucción y almacenamiento del operando de destino y **(4)** Comprobación de interrupciones.

(1) Lectura de la instrucción

Cada una de las operaciones que hacemos durante la ejecución de una instrucción la denominamos **microoperación**, y estas microoperaciones son la base para diseñar la unidad de control.

La fase de lectura de la instrucción consta básicamente de cuatro pasos:

- 1) MAR $\leftarrow$ PC:** se pone el contenido del registro PC en el registro MAR.
- 2) MBR $\leftarrow$ Memoria:** se lee la instrucción.
- 3) PC $\leftarrow$ PC + Δ :** se incrementa el PC tantas posiciones de memoria como se han leído (Δ posiciones).
- 4) IR $\leftarrow$ MBR:** se carga la instrucción en el registro IR.

(2) Lectura de los operandos fuente

El número de pasos que hay que hacer en esta fase depende del número de operandos fuente y de los modos de direccionamiento utilizados en cada operando. Si hay más de un operando, hay que repetir el proceso para cada uno de los operandos.

El modo de direccionamiento indica el lugar en el que está el dato:

- Inmediato: el dato está en la misma instrucción y, por lo tanto, no hay que hacer nada: IR (operando).
- Directo a registro: el dato está en un registro y, por lo tanto, no hay que hacer nada.
- Relativo a registro índice: el dato está en la memoria y hay que llevarlo al registro MBR.
 - o **MAR $\leftarrow$ IR(Dirección operando) + Contenido IR(registro índice)**
 - o **MBR $\leftarrow$ Memoria**: leemos el dato.

(3) Ejecución de la instrucción y almacenamiento del operando de destino

El número de pasos que hay que realizar en esta fase depende del código de operación de la instrucción y del modo de direccionamiento utilizado para especificar el operando de destino. Se necesita, por lo tanto, una descodificación para obtener esta información.

(4) Comprobación de interrupciones

En esta fase, si no se ha producido ninguna petición de interrupción, no hay que ejecutar ninguna microoperación y se continúa con la ejecución de la instrucción siguiente; en el caso contrario, hay que hacer un cambio de contexto. Para hacer un cambio de contexto hay que guardar el estado del procesador (generalmente en la pila del sistema) y poner en el PC la dirección de la rutina que da servicio a esta interrupción. Este proceso puede variar mucho de una máquina a otra. Aquí solo presentamos la secuencia de microoperaciones para actualizar el PC.

- **MBR $\leftarrow$ PC**: se pone el contenido del PC en el registro MBR.
- **MAR $\leftarrow$ Dirección de salvaguarda**: se indica dónde se guarda el PC.
- **Memoria $\leftarrow$ MBR**: se guarda el PC en la memoria.
- **PC $\leftarrow$ Dirección de la rutina**: se posiciona el PC al inicio de la rutina de servicio de la interrupción.

Tipos de unidades de control

Podemos distinguir 2 tipos de unidades de control:

- **Unidad de control CABLEADA**: es un circuito combinatorial que recibe un conjunto de señales de entrada y lo transforma en un conjunto de señales de salida que son las señales de control. Esta técnica es la que se utiliza típicamente en máquinas RISC.
- **Unidad de control MICROPROGRAMADA**: cada instrucción se compone de un conjunto de microinstrucciones denominado microprograma.

Las ventajas de la unidad de control microprogramada respecto de la cableada son:

- Simplifica el diseño.
- Es más económica.
- Es más flexible debido a que es:
 - o Adaptable a la incorporación de nuevas instrucciones.

- Adaptable a cambios de organización, tecnológicos, etc.
- Permite disponer de un juego de instrucciones con gran número de instrucciones. Solo hay que poseer una memoria de control de gran capacidad.
- Permite disponer de varios juegos de instrucciones en una misma máquina (emulación de máquinas).
- Permite ser compatible con máquinas anteriores de la misma familia.
- Permite construir máquinas con diferentes organizaciones pero con un mismo juego de instrucciones.

Los inconvenientes son:

- Es más lenta, ya que implica acceder a una memoria e interpretar las microinstrucciones necesarias para ejecutar cada instrucción.
- Es necesario un entorno de desarrollo para los microprogramas.
- Es necesario un compilador de microprogramas.

ALU (UNIDAD ARITMÉTICO LÓGICA)

Ejecuta las operaciones aritméticas (suma, resta, multiplicación, división) y lógicas (comparación, complementación, suma y producto lógico) que le señala la instrucción residente en la unidad de control.

La ALU es un circuito combinacional capaz de realizar operaciones aritméticas y lógicas con números enteros y también con números reales. Las operaciones que puede efectuar vienen definidas por el conjunto de instrucciones aritméticas y lógicas de las que dispone el juego de instrucciones del computador.

Veamos primero cómo se representan los valores de los números enteros y reales con los que puede trabajar la ALU y, a continuación, cuáles son las operaciones que puede hacer.

- 1) **Números enteros.** Los números enteros se pueden representar utilizando diferentes notaciones, entre las cuales hay signo magnitud, complemento a 1 y complemento a 2. La notación más habitual de los computadores actuales es el complemento a 2.

Todas las notaciones representan los números enteros en binario. Según la capacidad de representación de cada computador, se utilizan más o menos bits. El número de bits más habitual en los computadores actuales es de 32 y 64.

- 2) **Números reales.** Los números reales se pueden representar básicamente de dos maneras diferentes: en punto fijo y en punto flotante. El computador es capaz de trabajar con números reales representados en cualquiera de las dos maneras.

En la **notación en punto fijo** la posición de la coma binaria es fija y se utiliza un número concreto de bits tanto para la parte entera como para la parte decimal.

En la **notación en punto flotante** se representan utilizando tres campos, esto es: signo, mantisa y exponente, donde el valor del número es $\pm \text{mantisa} \cdot 2^{\text{exponente}}$.

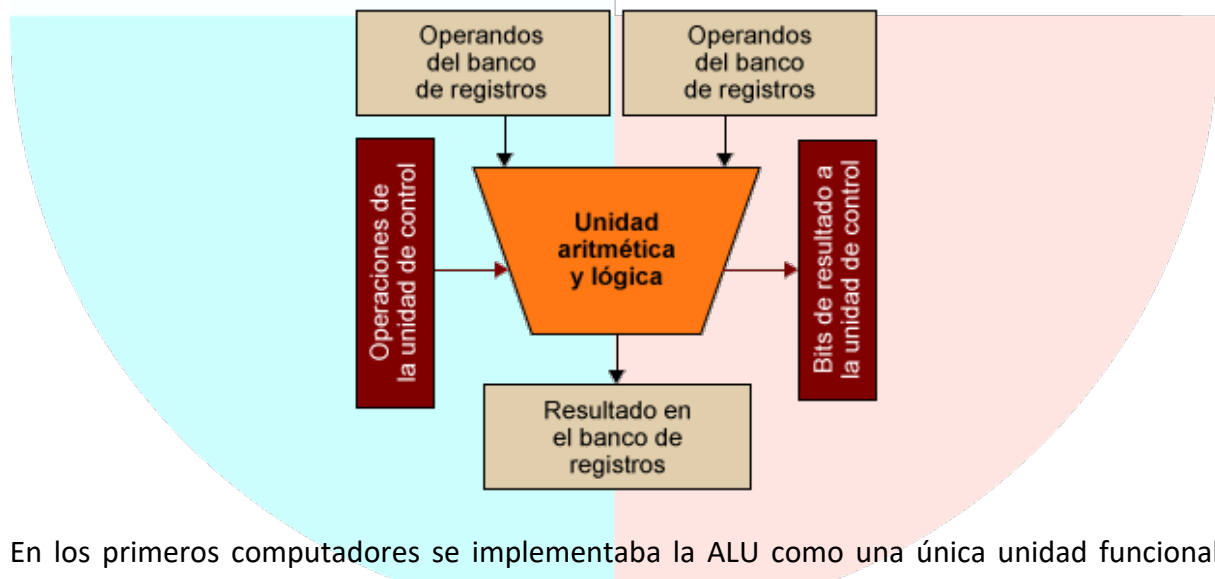
El IEEE ha definido una norma para representar números reales en punto flotante: IEEE-754. La norma define diferentes formatos de representación de números binarios en punto flotante. Los más habituales son los siguientes:

- Precisión simple: números binarios en punto flotante de 32 bits, utilizan un bit de signo, 8 bits para el exponente y 23 para la mantisa.
- Doble precisión: números binarios en punto flotante de 64 bits, utilizan un bit de signo, 11 bits para el exponente y 52 para la mantisa.
- Cuádruple precisión: números binarios en punto flotante de 128 bits, utilizan un bit de signo, 15 bits para el exponente y 112 para la mantisa.

La norma define también la representación del cero y de valores especiales, como infinito y NaN (Not a Number).

Las operaciones aritméticas habituales que puede hacer una ALU incluyen suma, resta, multiplicación y división. Además, se pueden incluir operaciones específicas de incremento positivo (+1) o negativo (-1).

Dentro de las operaciones lógicas se incluyen operaciones AND, OR, NOT, XOR, operaciones de desplazamiento de bits a la izquierda y a la derecha y operaciones de rotación de bits.



En los primeros computadores se implementaba la ALU como una única unidad funcional capaz de hacer las operaciones descritas anteriormente sobre números enteros. Esta unidad tenía acceso a los registros donde se almacenaban los operandos y los resultados de cada operación.

Para hacer operaciones en punto flotante, se utilizaba una unidad específica denominada unidad de punto flotante (FPU) o coprocesador matemático, que disponía de sus propios registros y estaba separada del procesador.

La evolución de los procesadores que pueden ejecutar las instrucciones solapadamente ha llevado a que el diseño de la ALU sea más complejo, de manera que ha hecho necesario replicar las unidades de trabajo con enteros para permitir ejecutar varias operaciones aritméticas en paralelo y, por otra parte, las unidades de punto flotante continúan implementando en unidades separadas pero ahora dentro del procesador.

REGISTROS DE LA CPU

Son "lugares" de almacenamiento temporal ubicados en la CPU. Contiene datos que esperan ser procesados por cualquier instrucción o datos que ya han sido tratados.

Almacenan temporalmente resultados y datos. Podemos destacar los siguientes registros:

- Registro Contador de Programa (PC): contiene la dirección de memoria de la siguiente instrucción a ejecutar.
- Registro de instrucción (IR): contiene la instrucción que se está ejecutando en ese momento. Entre las partes de una instrucción destaca el código de operación y las direcciones de memoria donde se encuentran los operandos que pueda necesitar la instrucción.
- Registro de estado.
- Registro de direcciones de memoria (Memory Access Register o MAR).
- Registro de datos de memoria (Memory buffer register o MBR).

MP (MEMORIA PRINCIPAL)

Almacena las instrucciones de los programas, los datos que ha de procesar y los resultados que obtiene la ejecutar los programas. Está constituida por celdas o elementos capaces de almacenar 1 bit de información (elemento de memoria que tiene el estado 0 o el estado 1). La memoria se organiza en conjuntos de elementos de un tamaño determinado y, a cada palabra, le corresponde una dirección única.

ENTRADA/SALIDA

Las instrucciones que ejecuta el computador y los datos necesarios para cada instrucción están almacenados en la memoria principal, pero para introducirlos en la memoria es necesario un dispositivo de entrada. Una vez ejecutadas las instrucciones de un programa y generados unos resultados, estos resultados se deben presentar a los usuarios y, por lo tanto, es necesario algún tipo de dispositivo de salida.

BUSES

Son los canales de comunicación entre los distintos elementos del computador. Un bus es un conjunto de líneas eléctricas (cada línea transmite 1 bit) que permiten transmitir direcciones,

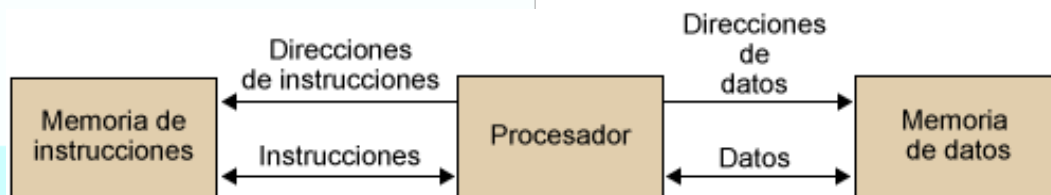
datos y señales de control entre dichos componentes. El ancho del bus representa el tamaño con el que trabaja el computador.

Existen 3 tipos de buses principales:

- De DATOS: son las líneas de comunicación por donde circulan los datos y las instrucciones que llegan a la CPU desde la memoria (lectura) o que llegan a la memoria desde la CPU (escritura).
- De DIRECCIÓN: designa la posición/dirección de la fuente o destino de un dato. Su anchura determina la máxima capacidad de memoria del sistema.
- De CONTROL: controlan el acceso y uso de los buses anteriores.

3. Arquitectura Harvard

La organización del computador según el modelo Harvard, básicamente, se distingue del modelo Von Neumann por la división de la memoria en **1 memoria de instrucciones y 1 memoria de datos**, de manera que el procesador puede acceder separada y simultáneamente a las dos memorias.



El procesador dispone de un sistema de conexión independiente para acceder a la memoria de instrucciones y a la memoria de datos. Cada memoria y cada conexión pueden tener características diferentes; por ejemplo, el tamaño de las palabras de memoria (el número de bits de una palabra), el tamaño de cada memoria y la tecnología utilizada para implementarlas.

Debe haber un mapa de direcciones de instrucciones y un mapa de direcciones de datos separados.

La arquitectura Harvard no se utiliza habitualmente en computadores de propósito general, sino que se utiliza en computadores para aplicaciones específicas, como:

- Microcontroladores: sistemas que controla el funcionamiento de un dispositivo (ej.: móviles, IoT).
- DSP (Digital Signal Processor o procesador de señales digitales): dispositivo capaz de procesar en tiempo real señales procedentes de diferentes fuentes (aplicaciones: procesamiento/reconocimiento de voz, procesamiento imágenes/audio digital).

4. Taxonomía de Flynn

Clasificación clásica de computadores en función de su arquitectura, publicada por Flynn por primera vez en 1966 y por segunda vez en 1970.

Esta taxonomía se basa en el flujo que siguen los datos dentro de la máquina y de las instrucciones sobre esos datos.

Se define como **flujo de instrucciones** al conjunto de instrucciones secuenciales que son ejecutadas por un único procesador.

Definimos **flujo de datos** al flujo secuencial de datos requeridos por el flujo de instrucciones. Con estas consideraciones, Flynn clasifica los sistemas en cuatro categorías:

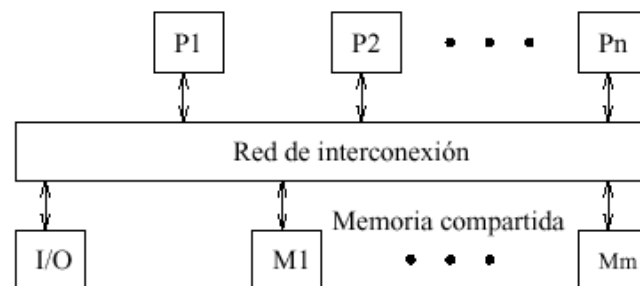
- **SISD** (Single Instruction stream, Single Data stream): los sistemas de este tipo se caracterizan por tener un único flujo de instrucciones sobre un único flujo de datos, es decir, se ejecuta una instrucción detrás de otra. Este es el concepto de **arquitectura serie de Von Neumann** donde, en cualquier momento, sólo se ejecuta una única instrucción. Esta arquitectura tanto el bus de direcciones como el bus de datos son simples. Las soluciones SISD NO presentan capacidades de concurrencia ni paralelización. En todo caso, podemos llegar a una simulación de estas capacidades, por medio de la segmentación.
- **SIMD** (Single Instruction stream, Multiple Data streams): estos sistemas tienen un único flujo de instrucciones que operan sobre múltiples flujos de datos. Ejemplos de estos sistemas los tenemos en las **máquinas vectoriales** con hardware escalar y vectorial. El procesamiento es síncrono, la ejecución de las instrucciones sigue siendo secuencial como en el caso anterior, todos los elementos realizan la misma instrucción pero sobre una gran cantidad de datos. Por este motivo existirá **concurrencia real** de operación, es decir, esta clasificación es el origen de la máquina paralela.
- **MISD** (Multiple Instruction streams, Single Data stream): sistemas con múltiples instrucciones que operan sobre un único flujo de datos. Este tipo de sistemas no ha tenido implementación hasta hace poco tiempo. Los sistemas MISD se contemplan de dos maneras distintas:
 - o Varias instrucciones operando simultáneamente sobre un único dato.
 - o Varias instrucciones operando sobre un dato que se va convirtiendo en un resultado que será la entrada para la siguiente etapa. Se trabaja de forma segmentada, todas las unidades de proceso pueden trabajar de forma concurrente.

Inconveniente: latencia de los datos. Se trata de arquitecturas experimentales. Ejemplos de estos tipos de sistemas son los **arrays sistólicos** (el movimiento de avance del dato es el 'latido' del sistema) o arrays de procesadores. También podemos encontrar aplicaciones de redes neuronales en máquinas masivamente paralelas.
- **MIMD** (Multiple Instruction streams, Multiple Data streams): sistemas con un flujo de múltiples instrucciones que operan sobre múltiples datos. Estos sistemas empezaron a utilizarse a principios de los 80. Se trata de sistemas basados en un único bus multiplexado, de altas prestaciones, a través del cual podemos transmitir simultáneamente más de una instrucción y más de un dato. Podemos distinguir dos categorías dentro de estos sistemas:
 - o **SMP** (Simetric MultiProcessor) o **MMC** (Multiprocesador de Memoria Compartida): tenemos **una única memoria compartida** donde se encuentran almacenadas todas las instrucciones del programa y los datos necesarios para ejecutar dichas instrucciones.

En estos sistemas existe un cuello de botella en el acceso a la memoria compartida por parte de los procesadores para acceder a datos globales o a instrucciones del programa. Se plantean dos alternativas para gestionar la memoria compartida en este tipo de sistemas:

- Gestión UMA (Uniform Memory Access): sistema multiprocesador con acceso uniforme a memoria. La memoria física es uniformemente compartida por todos los procesadores, esto quiere decir que todos los procesadores tienen el mismo tiempo de acceso a todas las palabras de la memoria. Cada procesador tiene su propia caché privada y también se comparten los periféricos.

De esta forma, si existen n procesadores, a cada uno le corresponderá $1/n$ del tiempo total de acceso a memoria.



- Gestión NUMA (Not Uniform Memory Access): se introducen criterios de preferencia en el acceso a la memoria compartida. Estos criterios pueden estar basados, por ejemplo, en la disposición física de los procesadores, o en una distribución de prioridades dada.
- MPP (Massively Parallel Processor) o MMD (Multiprocesador de Memoria Distribuida): estos sistemas surgen para solucionar la carencia de los anteriores, en cuanto a los accesos a la memoria compartida. En estos sistemas no existe un módulo de memoria compartida propiamente dicho, sino que ésta se encuentra distribuida en submódulos de memoria en cada uno de los procesadores individuales. Además, se añade un submódulo de E/S a los procesadores, que permita gestionar los accesos al submódulo de memoria interno. Esta arquitectura nos permite reducir considerablemente los accesos al bus para recuperar datos, siempre que aseguremos que las instrucciones del programa y los datos se encuentran distribuidos en los submódulos de memoria, agrupados de forma que en cada submódulo tengamos almacenados un grupo de instrucciones y todos los datos necesarios para ejecutarlas.

5. Arquitectura CISC vs RISC

En el diseño del procesador hay que tener en cuenta diferentes principios y reglas, con vistas a obtener un procesador con las mejores prestaciones posibles.

Las dos alternativas principales de diseño de la arquitectura del procesador son las siguientes:

- **Computadores CISC** (*Complex Instruction Set Computer*, computador con un juego de instrucciones **complejo**).
- **Computadores RISC** (*Reduced Instruction Set Computer*, computador con un juego de instrucciones **reducido**).

CISC

Características:

- El formato de instrucción es de longitud variable.
- Dispone de un gran juego de instrucciones, habitualmente más de cien, para dar respuesta a la mayoría de las necesidades de los programadores.
- Dispone de un número muy elevado de modos de direccionamiento.
- Es una familia anterior a la de los procesadores RISC.
- La unidad de control es microprogramada, es decir, la ejecución de instrucciones se realiza descomponiendo la instrucción en una secuencia de microinstrucciones muy simples.
- Procesa instrucciones largas y de tamaño variable, lo que dificulta el procesamiento simultáneo de instrucciones.

RISC

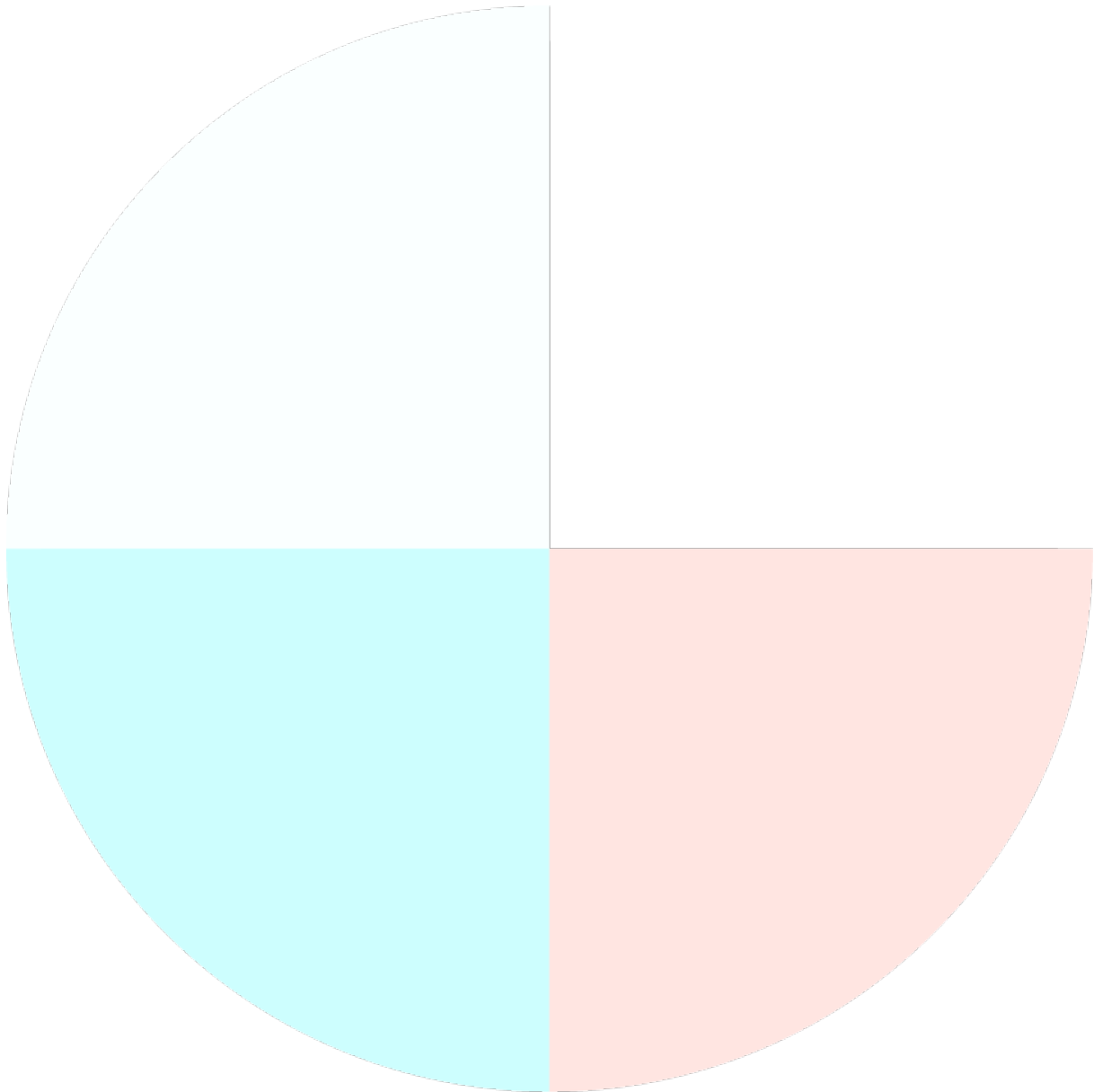
Características:

- El formato de instrucción es de tamaño fijo y corto, lo que permite un procesamiento más fácil y rápido.
- El juego de instrucciones se reduce a instrucciones básicas y simples, con las que se deben implementar todas las operaciones complejas. Una instrucción de un procesador CISC se tiene que escribir como un conjunto de instrucciones RISC.
- Dispone de un número muy reducido de modos de direccionamiento.
- La arquitectura es de tipo *load-store* (carga y almacena) o registro-registro. Las únicas instrucciones que tienen acceso a memoria son LOAD y STORE, y el resto de las instrucciones utilizan registros como operandos.
- Dispone de un amplio banco de registros de propósito general.
- Casi todas las instrucciones se pueden ejecutar en pocos ciclos de reloj.
- Este tipo de juego de instrucción facilita la segmentación del ciclo de ejecución, lo que permite la ejecución simultánea de instrucciones.
- La unidad de control es cableada.

Los procesadores actuales no son completamente CISC o RISC. Los nuevos diseños de una familia de procesadores con características típicamente CISC incorporan características RISC, de la misma manera que las familias con características típicamente RISC incorporan características CISC.

Arquitectura ARM

Es una arquitectura avanzada para microprocesadores RISC. Destacan por su bajo consumo energético. Uso generalizado en dispositivos móviles.



5. COMPONENTES INTERNOS DE LOS EQUIPOS MICROINFORMÁTICOS

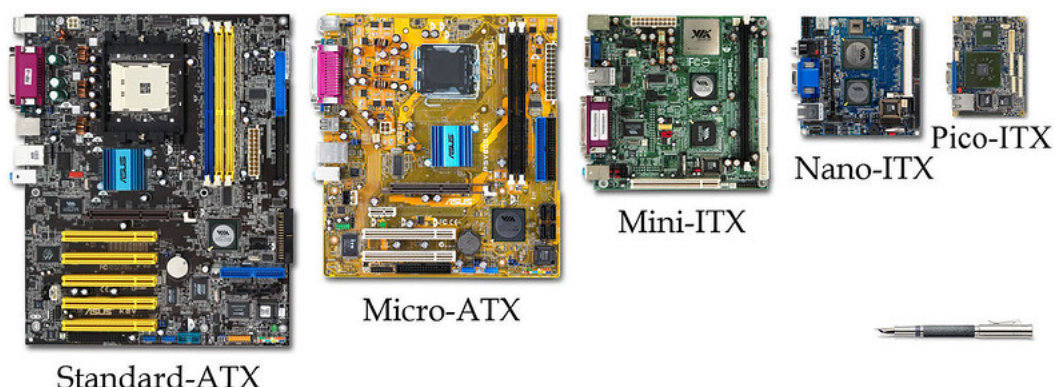
1. Placa base

La **placa base** o placa madre (mother board) de un ordenador es el dispositivo sobre el que se insertan los demás componentes del PC, tales como el microprocesador, las diferentes tarjetas de expansión y la memoria. La función principal de la placa base es servir de vía de comunicación entre los citados componentes, proporcionando las líneas eléctricas necesarias y las señales de control para que todas las transferencias de datos se lleven a cabo de manera rápida y fiable.

Se diseña básicamente para realizar tareas específicas vitales para el funcionamiento del computador, como por ejemplo:

- Conexión física.
- Administración, control y distribución de energía eléctrica.
- Comunicación de datos.
- Temporización.
- Sincronismo.
- Control y monitorización.

Para que la placa base cumpla con su cometido, tiene instalado un software muy básico denominado BIOS (Basic Input Output System), ahora UEFI. Este software es almacenado en un chip de memoria ROM, normalmente EPROM (Erasable Programmable Read-Only Memory), cuyo contenido permanece inalterable al apagar el PC. Se puede reprogramar y es el primer software en ejecutarse en el proceso de arranque de una placa base.



Cabe destacar, el concepto **factor de forma**, el cual define y estandariza un conjunto de características físicas de las placas base, como la forma, las dimensiones o las conexiones eléctricas de la fuente de alimentación.

Los diferentes formatos de placas base, ordenados cronológicamente, son:

- **XT** (eXtended Technology): tamaño A4, usado por IBM, años ochenta.
- **AT** (Advanced Techonology): tamaño 305 x 279-330 mm (12" x 11"- 13"). Usado hasta mediados de los años noventa.
- **ATX** (Advanced Technology eXtended): es la más extendida y su uso es variado, para uso personal, profesional o gaming. Conector de energía de 24 pines y conexiones exteriores. Tamaño 305 x 244 mm (12" x 9,6"). Variantes:
 - **eATX** (Extended ATX): tamaño 12" x 13". Utilizado en servidores.
 - **MicroATX**: tamaño máximo 244 x 244 mm (9,6" x 9,6"). Enfocado a equipos de oficina.
 - **FlexATX**: tamaño 229 x 191 mm (9,0" x 7,5").
 - **Mini ATX**: tamaño 284 x 208 mm (11,2" x 8,2"). Pensado para miniPCs o equipos de pequeñas dimensiones.
- **ITX** (Integrated Technology eXtended): evolución de MicroATX que permite integrar un número mayor de componentes. Variantes:
 - **Mini-ITX**: tamaño 170 x 170 mm (6,7" x 6,7").
 - **Nano-ITX**: tamaño 120 x 120 mm (4,7" x 4,7").
 - **Pico-ITX**: tamaño 100 x 72 mm (3,9" x 2,8").
- **BTX** (Balanced Technology Extended): evolución de ATX. Tamaño 325 x 266 mm (12,8" x 10,5"). Variantes:
 - **Micro BTX**: tamaño 264 x 267 mm (10,4" x 10,5").
 - **Pico BTX**: tamaño 203 x 267 mm (8,0" x 10,5").
 - **Regular BTX**: 325 x 267 mm (12,8" x 10,5").
- **DTX**: enfocado a miniPCs. Conector energía 24 pines. Tamaño 248 x 203 mm (9,8" x 8,0"). Variantes:
 - **Mini DTX**: tamaño 170 x 203 mm (6,7" x 8,0").
 - **Full DTX**: tamaño 243 x 203 mm (9,6" x 8,0").

Los factores de forma más comunes son: ATX, eATX, MicroATX y Mini-ITX (señalados con color azul).

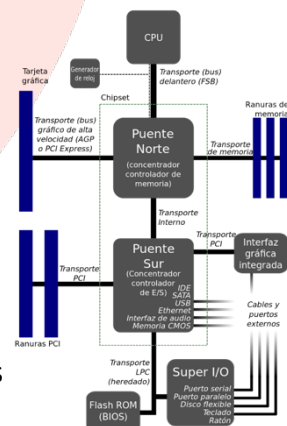
Factor de forma	Tamaño (mm)	Tamaño (pulgadas)
AT	305 x 279-330	12 x 11-13
ATX	305 x 244	12 x 9,6
eATX	305 x 330	12 x 13
MicroATX	244 x 244	9,6 x 9,6
FlexATX	229 x 191	9,0 x 7,5
MiniATX	284 x 208	11,2 x 8,2
Mini-ITX	170 x 170	6,7 x 6,7
Nano-ITX	120 x 120	4,7 x 4,7
Pico-ITX	100 x 72	3,9 x 2,8
BTX	325 x 266	12,8 x 10,5
Micro BTX	264 x 267	10,4 x 10,5
Pico BTX	203 x 267	8,0 x 10,5
Regular BTX	325 x 267	12,8 x 10,5
DTX	248 x 203	9,8 x 8,0
Mini DTX	170 x 203	6,7 x 8,0
Full DTX	243 x 203	9,6 x 8,0

(1) Chipset

Es un conjunto de circuitos integrados cuya función consiste en coordinar la transferencia de datos entre los distintos componentes del ordenador.

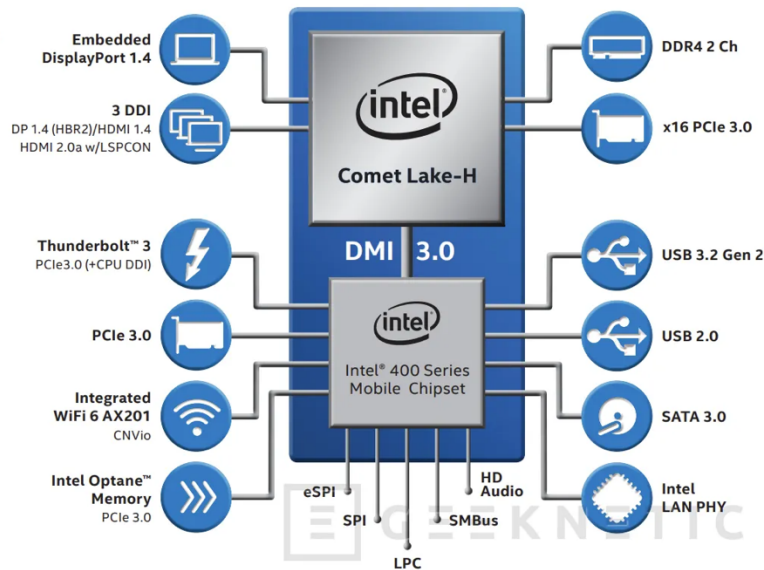
El diseño clásico de un chipset se dividía en:

- **Puente norte** (Northbridge): gestiona la interconexión entre el microprocesador, la memoria RAM y la unidad de procesamiento gráfico. Además se conecta con el Southbridge.
- **Puente sur** (Southbridge): gestiona la interconexión entre los periféricos y los dispositivos de almacenamiento.

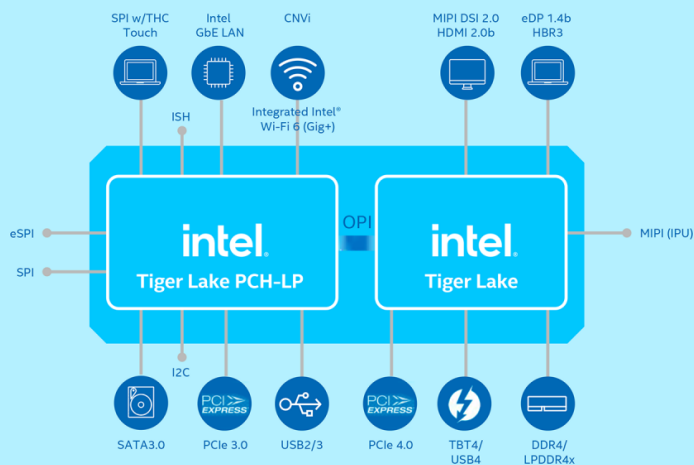


Por requisitos de rendimiento, el Northbridge fue absorbido en la propia CPU y conectado punto a punto con el Southbridge. Esta arquitectura es la denominada, según Intel, PCH (Platform Controller Hub) existente en la actualidad. El Southbridge cambia de nombre a ICH (I/O Controller Hub).

10TH GEN INTEL® CORE™ MOBILE PROCESSOR OVERVIEW



11th Gen Intel® Core™ Processor



(2) BIOS

La BIOS (Sistema básico de entrada y salida) es un componente de la placa base que se encuentra almacenado en un módulo de memoria de tipo ROM. Se encarga de verificar (chequear) el funcionamiento del hardware. Para ello, utiliza los datos almacenados en la pila CMOS para buscar la configuración del hardware del sistema. Tras el test realizado, carga un gestor de arranque o el sistema operativo. **UEFI** (Unified Extensible Firmware Interface) es el sucesor de la BIOS.

(3) Zócalo para el procesador

Existen dos categorías de soporte:

- **Slot:** zócalo de microprocesador para inserción vertical.
- **Socket:** zócalo de microprocesador cuadrangular o rectangular.

Tipos de sockets:

- **LGA (Land Grid Array)**: tiene los pines de contacto incorporados directamente a la propia estructura del socket. Ejemplos: LGA 3647, LGA 1151, LGA 2066, LGA 1200 (2020), LGA 1700 (2021). El número indica la cantidad de pines.
- **PGA (Pin Grid Array)**: se caracteriza porque los pines de contacto están incorporados al procesador y no en la estructura del socket. A la inversa que en el caso anterior. Ejemplos: Socket AM4 (PGA 1331), Socket TR4.
- **BGA (Ball Grid Array)**: no se utilizan pines. El procesador se une a la placa base mediante pequeñas bolas soldadas. Esto impide el cambio de procesador. Usado en portátiles y dispositivos móviles. Ventajas: mayor densidad de contactos, menor ocupación de espacio y conduce mejor el calor.



Para insertar los procesadores con mayor facilidad, se ha creado un concepto llamado **ZIF** (Fuerza de inserción nula). Los zócalos ZIF poseen una pequeña palanca que, cuando se levanta, permite insertar el procesador sin aplicar presión. Al bajarse, esta mantiene el procesador en su lugar.

(4) Ranuras de memoria RAM

Pueden ser de tipo SIMM, DIMM o SODIMM.

(5) Ranuras de expansión

Existen varios tipos de ranuras:

- **ISA** (Arquitectura estándar industrial): permiten insertar ranuras ISA.
- **PCI** (Interconexión de componentes periféricos): se utilizan para conectar tarjetas PCI.
- **PCI Express** o **PCIe** (Interconexión de componentes periféricos rápida).
- **AGP** (Puerto gráfico acelerado): es un puerto rápido para tarjetas gráficas.
- **M.2**: para conectar discos SSD.
- **U.2**: similar a M.2, pero ocupa más espacio en la placa base.
- **SATA** (serial ATA): utilizado para conexión de discos duros de 2,5" o 3,5".

(6) Reloj de Tiempo Real (RTC)

Su función es la de sincronizar las señales del sistema.

(7) Pila CMOS

Su función es doble: por un lado, se encarga de mantener la alimentación eléctrica del RTC y, por otro, almacena algunos datos del sistema, como la hora, la fecha y algunas configuraciones esenciales del sistema.

(8) Conectores de entrada y salida

Podemos enumerar los siguientes:

- Puerto serie.
- Puerto paralelo.
- Puerto USB.
- Conectores IDE, ATA, SATA, SCSI, SAS.
- Conector RJ45.
- Conectores VGA, DVI, HDMI.
- Conectores de audio.

(9) Front Side Bus (FSB)

Este bus representa el camino por el cual es posible integrar en la placa base los distintos componentes hardware para el intercambio de información entre microprocesador, memoria y el subsistema de Entrada/Salida (subsistema de E/S).

Para realizar intercambio de información con cualquiera de los elementos microprocesador, memoria y subsistema de E/S se debe hacer uso del FSB. La velocidad del FSB es siempre inferior a la velocidad interna del microprocesador.

En todo PC hay que diferenciar dos frecuencias distintas:

- la frecuencia interna o velocidad de ciclo del procesador y
- la frecuencia externa o velocidad del FSB.

Hay que ajustar velocidades de los distintos componentes hardware. El elemento que introduce esa posibilidad de ajuste entre distintas velocidades es el chipset.

La técnica que permite aumentar la frecuencia de reloj de un componente electrónico se denomina **overclocking**.

Pregunta de examen

De entre los siguientes, ¿cuál NO es un componente de la placa base de un ordenador?

- a) Zócalo del procesador.
- b) Chipset.
- c) Reloj.
- d) **Chip EAC.**

2. Microprocesadores

Conceptos básicos a tener en cuenta:

- **Proceso de fabricación:** el proceso de fabricación especifica cuál es el tamaño de los transistores integrados en el procesador cuando se fabrica. Este valor se expresa en nanómetros (nm) y se refiere a la medida de un transistor. Actualmente se están utilizando procesadores de 14, 10, 7, 5 y 3 (2024) nanómetros².
- **Número de núcleos (cores):** originalmente, los procesadores se fabricaban con un solo núcleo que hacía todo el trabajo, pero con el paso del tiempo se dieron cuenta de que uno solo no podía hacer todo el trabajo y gracias a la reducción de las litografías, pudieron integrar cada vez más núcleos dentro de los procesadores³.

Cada uno de los núcleos es en esencia un procesador en sí mismo, y permite que el conjunto pueda realizar muchas tareas paralelas (paralelismo).

A día de hoy podemos encontrar procesadores desde 4 a 32 núcleos, incluso más.

- **Número de hilos (threads):** el multithreading en un procesador (**Hyper-Threading** en Intel o SMT -Simultaneous Multi-Threading- en AMD) significa que cada uno de los núcleos físicos es capaz de realizar 2 tareas, es decir, ejecutar 2 hilos de proceso de manera simultánea (conurrencia). En definitiva, permite simular 2 núcleos lógicos para cada núcleo físico. Por lo tanto, un procesador de 4 núcleos físicos con multithreading tendría 8 hilos de proceso, y sería capaz de ejecutar 8 órdenes al mismo tiempo.

Intel

Tipos de procesadores familia Intel (ordenados de mayor a menor prestaciones):

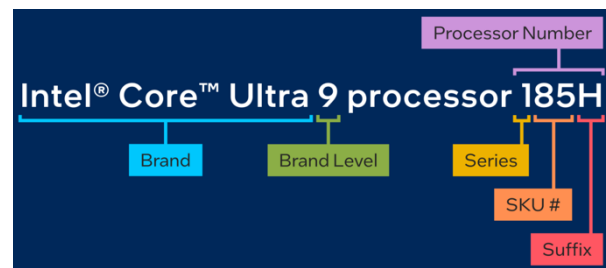
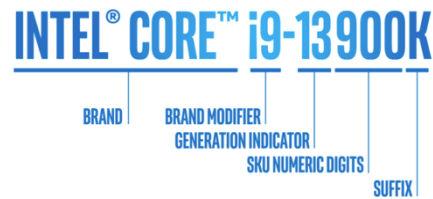
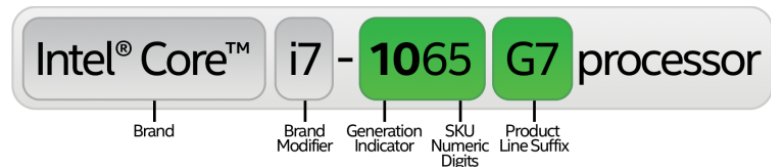
- **Xeon escalable:** Bronze, Platinum, Gold y Silver. Destinado a servidores de alto rendimiento.
- **Xeon:** E, D, W y serie Max. Destinado a servidores.
- **Core Ultra:** 5, 7 y 9. Destinado a equipos de sobremesa y portátiles, incorporan NPUs (unidades de procesamiento neuronal) para IA y GPU (Graphics Processing Unit) Intel Arc para gaming.
- **Core:** i3, i5, i7, i9 y X. Destinado a equipos de sobremesa y portátiles.
- **Celeron:** G, J y N. Destinado a equipos con pocas prestaciones y dispositivos móviles.
- **Atom:** C y P. Destinado a dispositivos móviles e IoT.

² Ley de Moore: El número de transistores de un chip se duplica cada dos años.

³ Ley de Amdahl: La mejora obtenida en el rendimiento de un sistema debido a la alteración de uno de sus componentes está limitada por la fracción de tiempo que se utiliza dicho componente.

Generaciones y nombre en clave procesadores Intel Core:

- 1ª generación: Nehalem.
- 2ª generación: Sandy Bridge.
- 3ª generación: Ivy Bridge.
- 4ª generación: Haswell.
- 5ª generación: Broadwell.
- 6ª generación: Skylake.
- 7ª generación: Kaby Lake.
- 8ª generación: Coffee Lake.
- 9ª generación: Coffee Lake Refresh.
- 10ª generación: Comet Lake.
- 11ª generación: Tiger Lake.
- 12ª generación: Alder Lake.
- 13ª generación: Raptor Lake.
- 14ª generación: Raptor Lake S.



Chips Apple

En mayo de 2021 Apple lanzó el nuevo chip M1, diseñado específicamente para Mac. Destacamos:

- **CPU de 8 núcleos**: 4 núcleos de rendimiento + 4 núcleos de eficiencia.
- **GPU de 8 núcleos**.
- **Neural Engine** (16 núcleos): aprendizaje automático avanzado.
- **16 billones de transistores**.
- Proceso de fabricación de **5 nm**.
- Rendimiento de **2,6 TFLOPS**.



En junio de 2022, Apple anunció el chip M2, evolución de M1 que ofrece, en general, un mayor rendimiento.

En octubre de 2023, Apple lanzó el chip M3, con mayor rendimiento, una CPU y un Neural Engine más rápidos, la compatibilidad con memoria unificada adicional. Modelos de M3: M3, M3 PRO y M3 MAX.

En mayo de 2024, se presentó el chip M4, con una CPU de 10 núcleos (4 de rendimiento y 6 de eficiencia), conectividad con Thunderbolt 5 y un proceso de fabricación de 3 nm.

En octubre 2024 se lanzaron los chips M4 PRO (14 núcleos, 10 de rendimiento y 4 de eficiencia; GPU de 20 núcleos) y M4 MAX (16 núcleos, 12 de rendimiento y 4 de eficiencia; GPU de 40 núcleos).

Rendimiento de un procesador

Por otro lado, en relación con el **rendimiento** teórico de un procesador podemos hacer uso de distintas medidas:

- **MIPS**: mide cuántos millones de instrucciones por segundo es capaz de ejecutar un procesador. Este índice no tiene en cuenta que hay procesadores con instrucciones más potentes que las de otros, por lo que se puede considerar como una medida simplista.
- **FLOPS**: mide el número de operaciones de coma flotante por segundo es capaz de ejecutar el procesador. Análogamente, esta medida no tiene en cuenta las diferencias entre las instrucciones de procesadores diferentes. Normalmente, los fabricantes de procesadores suelen proporcionar para los índices MIPS y MFLOPS picos de rendimiento.

MFLOPS	megaFLOPS = 10^6
GFLOPS	gigaFLOPS = 10^9
TFLOPS	teraFLOPS = 10^{12}
PFLOPS	petaFLOPS = 10^{15}
EFLOPS	exaFLOPS = 10^{18}

- **SPEC**: ejecuta en el procesador un conjunto de programas y combina las medidas de prestaciones de estos con la media aritmética o geométrica.
- **Productividad (throughput)**: número de tareas realizadas por unidad de tiempo.

3. Memoria interna

La memoria interna en un computador moderno está formada típicamente por dos niveles fundamentales: memoria caché y memoria principal. En los computadores actuales es frecuente encontrar la memoria caché también dividida en niveles.

Todos los tipos de memoria interna se implementan utilizando tecnología de semiconductores y tienen el transistor como elemento básico de su construcción.

El elemento básico en toda memoria es la celda. Una celda permite almacenar un bit, un valor 0 o 1 definido por una diferencia de potencial eléctrico. La manera de construir una celda de memoria varía según la tecnología utilizada.

La memoria interna es una memoria de acceso aleatorio, pudiendo acceder a cualquier palabra de memoria especificando una dirección de memoria.

Una manera de clasificar la memoria interna según la perdurabilidad es la siguiente:

- **Memoria VOLÁTIL:** es la memoria que necesita una corriente eléctrica para mantener su estado, de manera genérica denominada *RAM*.
 - **SRAM** (Static Random Access Memory): la memoria estática de acceso aleatorio implementa cada celda de memoria utilizando un flip-flop básico para almacenar un bit de información, y mantiene la información mientras el circuito de memoria recibe alimentación eléctrica. Para implementar cada celda de memoria son necesarios varios transistores, típicamente 6, por lo que la memoria tiene una capacidad de integración limitada y su coste es elevado en relación con otros tipos de memoria RAM, como la DRAM. Sin embargo, es el tipo de memoria RAM más rápido.
 - **DRAM** (Dynamic Random Access Memory): la memoria dinámica implementa cada celda de memoria utilizando la carga de un condensador. A diferencia de los flip-flops, los condensadores con el tiempo pierden la carga almacenada y necesitan un circuito de refresco para mantener la carga y mantener, por lo tanto, el valor de cada bit almacenado. Eso provoca que tenga un tiempo de acceso mayor que la SRAM. Cada celda de memoria está formada por solo un transistor y un condensador. Por lo tanto, las celdas de memoria son mucho más pequeñas que las celdas de memoria SRAM, lo que garantiza una gran escala de integración y al mismo tiempo permite hacer memorias más grandes en menos espacio.
 - **DRAM Asíncrona** (Asynchronous Dynamic Random Access Memory): memoria de acceso aleatorio dinámica asíncrona.
 1. **FPM RAM** (Fast Page Mode RAM).
 2. **EDO RAM** (Extended Data Output RAM).
 3. **BEDO RAM** (Burst Extended Data Output RAM).
 - **SDRAM** (Synchronous Dynamic Random Access Memory): memoria de acceso aleatorio dinámica síncrona. Tipos:
 1. **SDR SDRAM** (Single Data Rate Synchronous Dynamic Random Access Memory): SDRAM de tasa de datos simple, es decir, una operación en cada ciclo de reloj, módulo DIMM de 168 pines y módulo SODIMM de 144 pines.
 2. **DDR SDRAM** (Double Data Rate Synchronous Dynamic Random Access Memory): SDRAM de tasa de datos doble, es decir, dos operaciones en cada ciclo de reloj, módulo DIMM de 184 pines, módulo SODIMM de 200 pines.
 3. **DDR2 SDRAM** (Double Data Rate type two SDRAM): SDRAM de tasa de datos doble de tipo dos, módulo DIMM de 200 pines, módulo SODIMM de 200 pines.
 4. **DDR3 SDRAM** (Double Data Rate type three SDRAM): SDRAM de tasa de datos doble de tipo tres, módulo DIMM de 240 pines, módulo SODIMM de 204 pines.
 5. **DDR4 SDRAM** (Double Data Rate type four SDRAM): SDRAM de tasa de datos doble de tipo cuatro, módulo DIMM de 288 pines, módulo SODIMM de 260 pines.
 6. **DDR5 SDRAM** (Double Data Rate type five SDRAM): SDRAM de tasa de datos doble de tipo cinco, módulo DIMM de 288 pines.

DIMM (Dual Inline Memory Module): módulo de memoria con contactos independientes por las dos caras, dirigido a equipos de sobremesa.

SO-DIMM (Small Outline Dual Inline Memory Module): versión DIMM de menor tamaño dirigido a preferentemente a portátiles y miniPCs.

- **Memoria NO VOLÁTIL:** es la que mantiene el estado sin necesidad de corriente eléctrica.
 - **ROM** (Read Only Memory): la memoria de solo lectura solo lectura que no permiten operaciones de escritura y, por lo tanto, la información que contienen no se puede borrar ni modificar. Este tipo de memorias se pueden utilizar para almacenar los microprogramas en una unidad de control microprogramada, también se pueden utilizar en dispositivos que necesitan trabajar siempre con la misma información. La grabación de la información en este tipo de memorias forma parte del proceso de fabricación del chip de memoria. Estos procesos implican la fabricación de un gran volumen de memorias ROM con la misma información; es un proceso costoso y no es rentable para un número reducido de unidades.
 - **PROM** (Programmable Read Only Memory): la memoria programable de solo lectura es el usuario final el que puede grabar el contenido según sus necesidades. Cabe destacar que, tal como sucede con las memorias ROM, el proceso de grabación o programación solo se puede realizar una vez. Este tipo de memoria tiene unas aplicaciones parecidas a las de las memorias ROM.
 - **EPROM** (Erasable Programmable Read Only Memory): la memoria programable y borrrable de solo lectura es una memoria en la que habitualmente se hacen operaciones de lectura, pero cuyo contenido puede ser borrado y grabado de nuevo. Hay que destacar que el proceso de borrar es un proceso que borra completamente todo el contenido de la memoria, no se puede borrar solo una parte. Para borrar, se aplica luz ultravioleta sobre el chip de memoria EPROM. La grabación de la memoria se hace mediante un proceso eléctrico utilizando un hardware específico. Tanto para el proceso de borrar como para el proceso de grabar hay que sacar el chip de memoria de su localización de uso habitual, ya que la realización de estas dos tareas implica la utilización de hardware específico.
 - **EEPROM** (Electrically Erasable Programmable Read Only Memory): la memoria programable y borrrable eléctricamente de solo lectura tiene un funcionamiento parecido a la EPROM, permite borrar el contenido y grabar información nueva. Sin embargo, a diferencia de las memorias EPROM, todas las operaciones son realizadas eléctricamente. Para grabar datos no hay que borrarlos previamente, se permite modificar directamente solo uno o varios bytes sin modificar el resto de la información. Son memorias mayoritariamente de lectura, ya que el proceso de escritura es considerablemente más lento que el proceso de lectura.
- **Memoria FLASH:** es un tipo de memoria parecida a las memorias EEPROM, en las que el borrado es eléctrico, con la ventaja de que el proceso de borrar y grabar es muy rápido. La velocidad de lectura es superior a la velocidad de escritura, pero las dos son del mismo orden de magnitud.

Jerarquía de memorias

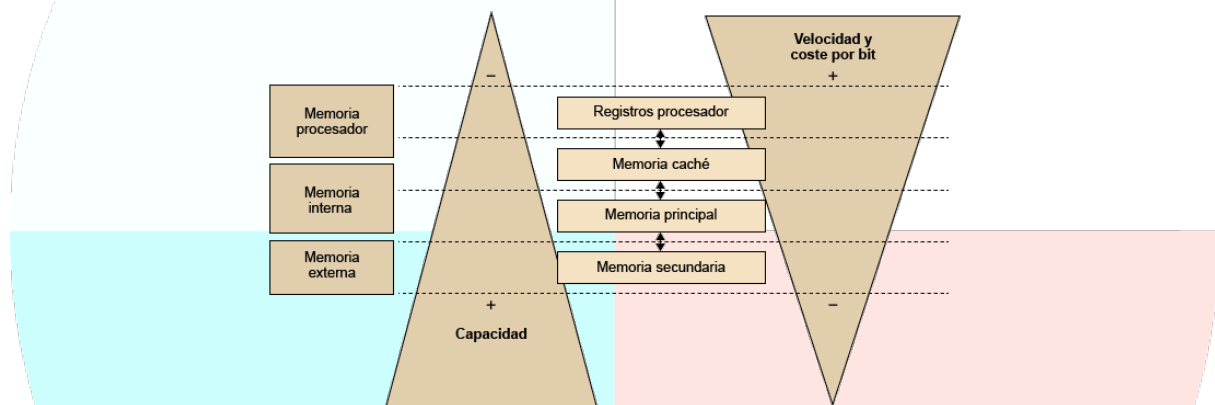
En una jerarquía de memorias se utilizan varios tipos de memoria con distintas características de capacidad, velocidad y coste, que dividiremos en niveles diferentes: memoria del procesador, memoria interna y memoria externa.

Cada nivel de la jerarquía se caracteriza también por la distancia a la que se encuentra del procesador. Los niveles más próximos al procesador son los primeros que se utilizan. Eso es así porque también son los niveles con una velocidad más elevada.

Propiedades de la jerarquía:

- **Inclusión:** la información almacenada en el nivel M_i debe encontrarse también en los niveles M_{i+1} , M_{i+2} , ..., M_n .
- **Coherencia:** si un bloque de información se modifica en el nivel M_i , deben actualizarse los niveles M_{i+1} , ..., M_n .

A continuación, se muestra cuál es la variación de la capacidad, velocidad y coste por bit para los niveles típicos de una jerarquía:



Memoria caché

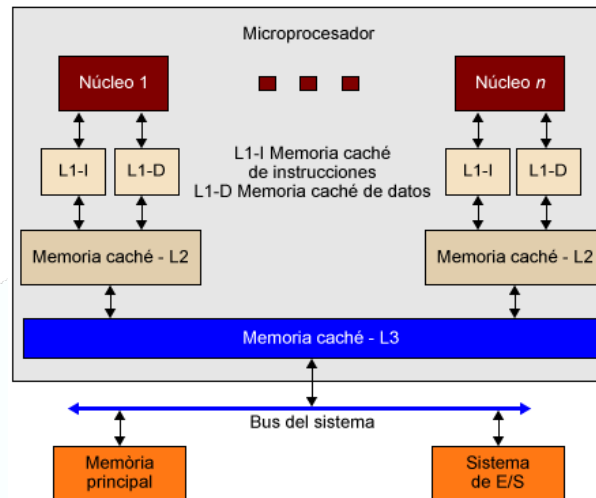
La **memoria caché** se sitúa entre la memoria principal y el procesador, puede estar formada por uno o varios niveles.

Son memorias de capacidad reducida, pero más rápidas que la memoria principal, que utilizan un método de acceso asociativo. Se pueden encontrar dentro del chip del procesador o cerca de él y están diseñadas para reducir el tiempo de acceso a la memoria. En la memoria caché se almacenan los datos que se prevé que se utilizarán más habitualmente, de manera que sea posible reducir el número de accesos que debe hacer el procesador a la memoria principal (ya que el tiempo de acceso a la memoria principal siempre es superior al tiempo de acceso a la memoria caché).

No es accesible por parte del programador, es gestionada por el hardware y el sistema operativo y se implementa utilizando **tecnología SRAM**.

Los procesadores modernos utilizan diferentes niveles de memoria caché, lo que se conoce como **memoria caché de 1º nivel, 2º nivel**, etc. Actualmente es habitual disponer de hasta 3

niveles de memoria caché, referidos como L1, L2 y L3. Cada vez es más frecuente que algunos de estos niveles se implementen dentro del chip del procesador y que el nivel más próximo al procesador esté dividido en 2 partes: 1 dedicada a las instrucciones y 1 dedicada a los datos.



- La **caché L1** es el primer lugar donde el microprocesador buscará información. Es la caché más pequeña y más rápida.
- La **caché L2** suele ser de mayor tamaño que la caché L1, pero es algo más lenta. Sin embargo, es la que mayor impacto tiene en el rendimiento.
- La **caché L3** es de mucho más tamaño que las anteriores, y generalmente se comparte entre todos los núcleos del procesador, a diferencia de las anteriores, que normalmente van ligadas a un core. Este tercer nivel es en el que buscará el procesador la información tras no encontrarla en la L1 y L2, por lo que su tiempo de acceso es todavía mayor.

Localidad de referencia: temporal y espacial

Los programas manifiestan una propiedad que se explota en el diseño del sistema de gestión de memoria de los ordenadores en general y de la memoria caché en particular, la localidad de referencias: los programas tienden a reutilizar los datos e instrucciones que utilizaron recientemente. Una regla empírica que se suele cumplir en la mayoría de los programas revela que gastan el 90% de su tiempo de ejecución sobre sólo el 10% de su código. Una consecuencia de la localidad de referencia es que se puede predecir con razonable precisión las instrucciones y datos que el programa utilizará en el futuro cercano a partir del conocimiento de los accesos a memoria realizados en el pasado reciente. La localidad de referencia se manifiesta en una doble dimensión: temporal y espacial.

- **Localidad temporal:** las palabras de memoria accedidas recientemente tienen una alta probabilidad de volver a ser accedidas en el futuro cercano. La localidad temporal de los programas viene motivada principalmente por la existencia de bucles.
- **Localidad espacial:** las palabras próximas en el espacio de memoria a las recientemente referenciadas tienen una alta probabilidad de ser también referenciadas en el futuro cercano. Es decir, que las palabras próximas en memoria tienden a ser referenciadas juntas en el tiempo. La localidad espacial viene motivada fundamentalmente por la linealidad de

los programas (secuencialidad de las instrucciones) y el acceso a las estructuras de datos regulares.

Política de asignación o estrategia de organización

El número de líneas disponibles en la memoria caché es siempre mucho más pequeño que el número de bloques de memoria principal. En consecuencia, la memoria caché, además de la información almacenada, debe mantener alguna información que relacione cada posición de la memoria caché con su dirección en la memoria principal (función de correspondencia).

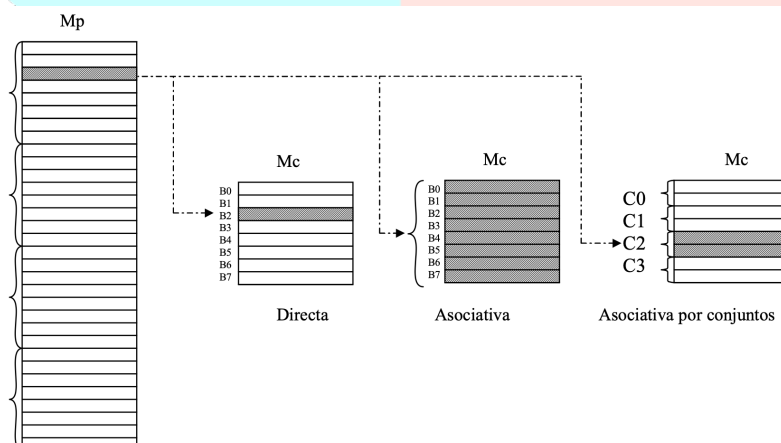
Para acceder a un dato se especifica la dirección en la memoria principal; a partir de esta dirección hay que verificar si el dato está en la memoria caché. Esta verificación se hace a partir del campo *etiqueta* de la línea de la memoria caché que indica qué bloque de memoria principal se encuentra en cada una de las líneas de la memoria caché.

La política de asignación determina dónde se puede colocar un bloque de la memoria principal dentro de la memoria caché y condiciona cómo encontrar un dato dentro de la memoria caché.

La organización de la memoria caché puede ser:

- **Directa:** un bloque de la memoria principal solo puede estar en una única línea de la memoria caché. Dirección memoria: **etiqueta + marco de bloque + palabra**.
- **Totalmente asociativa:** un bloque de la memoria principal puede estar en cualquier línea de la memoria caché. Dirección memoria: **etiqueta + palabra**.
- **Asociativa por conjuntos:** un bloque de la memoria principal puede estar en un subconjunto de las líneas de la memoria caché, pero dentro del subconjunto puede encontrarse en cualquier posición. Dirección memoria: **etiqueta + subconjunto (índice) + palabra**.

La memoria caché asociativa por conjuntos es una combinación de la memoria caché de asignación completamente asociativa y la memoria caché de asignación directa. El número de elementos de cada subconjunto no suele ser muy grande, un número habitual de elementos es entre 4 y 64. Si el número de elementos del subconjunto es n , la memoria caché se denomina n -asociativa.



Líneas de la memoria caché donde podemos asignar el bloque x según las diferentes políticas de asignación:

Dirección	Memoria principal	Bloque
0		Bloque 0
1		
...		
$k - 1$		
k		Bloque 1
$k + 1$		
...		
$2 \cdot k - 1$		
...	...	...
$x \cdot k + 0$		Bloque x
$x \cdot k + (k - 1)$		
...	...	...
$2^n - k$		Bloque $(2^n / k) - 1$
...		
$2^n - 1$		

Memoria caché de acceso directo		
Línea	Etiqueta	Palabras del bloque
0		
...		...
Línea asignada al bloque x		
...		...
$m - 1$		

Memoria caché de asignación completamente asociativa		
Línea		Contenido de la caché
0		
...	...	...
Bloque x asignado a cualquier línea		
...	...	...
$m - 1$		

Memoria caché de asignación asociativa por conjuntos		
Línea		Contenido de la caché
0		
...	...	...
...		
Conjunto de líneas asignado al bloque x		
...	...	...
$m - 1$		

Algoritmos de reemplazo

Cuando se produce un fallo y se tiene que llevar a la memoria caché un bloque de memoria principal determinado, si este bloque de memoria se puede almacenar en más de una línea de la memoria caché, hay que decidir en qué línea de todas las posibles se pone, y sobrescribir los datos que se encuentran en aquella línea. El algoritmo de reemplazo se encarga de esta tarea.

En una memoria caché directa no es necesario ningún algoritmo de reemplazo, ya que un bloque solo puede ocupar una única línea dentro de la memoria caché. En una memoria caché completamente asociativa, solo se aplica el algoritmo de reemplazo para seleccionar una de las líneas de la memoria caché. En una memoria caché asociativa por conjuntos, solo se aplica el algoritmo de reemplazo para seleccionar una línea dentro de un conjunto concreto.

Para que estos algoritmos de reemplazo no penalicen el tiempo medio de acceso a memoria, se deben implementar en hardware y, por lo tanto, no deberían ser muy complejos.

A continuación se describen de manera general los algoritmos de reemplazo más comunes, pero se pueden encontrar otros algoritmos o variantes de estos:

- **FIFO (First In First Out):** para elegir la línea se utiliza una cola, de manera que la línea que hace más tiempo que está almacenada en la memoria caché será la reemplazada. Este algoritmo puede reducir el rendimiento de la memoria caché porque la línea que se encuentra almacenada en la memoria caché desde hace más tiempo no tiene que ser necesariamente la que se utilice menos.

Se puede implementar fácilmente utilizando técnicas de buffers circulares (o round robin): cada vez que se debe sustituir una línea se utiliza la línea del buffer siguiente, y cuando se llega a la última, se vuelve a empezar desde el principio.

- **LFU (Least Frequently Used):** se elige la línea que se ha utilizado menos veces. Se puede implementar añadiendo un contador del número de accesos a cada línea de la memoria caché.
- **LRU (Least Recently Used):** elige la línea que hace más tiempo que no se utiliza. Es el algoritmo más eficiente, pero el más difícil de implementar, especialmente si hay que elegir entre muchas líneas. Se utiliza habitualmente en memorias caché asociativas por conjuntos, con conjuntos pequeños de 2 o 4 líneas.

Para memorias cachés 2-asociativas, se puede implementar añadiendo un bit en cada línea; cuando se hace referencia a una de las dos líneas, este bit se pone a 1 y el otro se pone a 0 para indicar cuál de las dos líneas ha sido la última que se ha utilizado.

- **Aleatorio:** los algoritmos anteriores se basan en factores relacionados con la utilización de las líneas de la caché; en cambio, este algoritmo elige la línea que se debe reemplazar al azar. Este algoritmo es muy simple y se ha demostrado que tiene un rendimiento solo ligeramente inferior a los algoritmos que tienen en cuenta factores de utilización de las líneas.

Memoria principal

En la memoria principal se almacenan los programas que se deben ejecutar y sus datos, es la memoria visible para el programador mediante su espacio de direcciones.

La memoria principal se implementa utilizando diferentes chips conectados a la placa principal del computador y tiene una capacidad mucho más elevada que la memoria caché (del orden de Gbytes o de Tbytes).

Utiliza tecnología DRAM, que es más lenta que la SRAM, pero con una capacidad de integración mucho más elevada, hecho que permite obtener más capacidad en menos espacio.

4. Proceso de arranque de un computador

Paso 1. Encendido del PC (power on)

Conectar la fuente de alimentación y suministrar corriente eléctrica a la placa y resto de dispositivos.

Paso 2. BIOS (UEFI)

El microprocesador ejecuta la BIOS, la cual toma el control y ejecuta el POST.

Paso 3. POST

El POST (Power On Self Test) hace un chequeo de todos los componentes.

Paso 4. Inicio adaptador de vídeo y habilitación de dispositivos

Paso 5. Carga del gestor de arranque (boot manager) y cede el control al sistema operativo

